

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



INTELLIGENT SYSTEM DEVELOPMENT
ASSIGNMENT 02

From Data Representation to Deployable Intelligent Systems

Student name:	Nguyễn Văn Trường
Student ID:	B23DCCE095
Class:	E23CNPM02
Lecturer:	Dinh Que Tran, Ph.D., Assoc. Prof.
Semester:	I.2026

Table of Contents

1. Introduction and Objectives	3
2. Connection to Lecture 02 - Data Representation	4
3. Data Sources and Application Definitions	4
4. Application 1 - Diabetes Prediction	5
5. Application 2 - House-Price Prediction.....	11
6. Application 3 - E-Commerce Customer Behaviour and Interest	16
7. Comparison of the Three Intelligent Systems.....	25
8. Deployment Architecture.....	26
9. Reproducibility.....	28
10. Final Comparative Discussion	29
11. Conclusion	30
Appendix A - Repository Structure.....	30
Appendix B — Jupyter Notebook Architecture & Section Specification.....	31
Appendix C - API Endpoint Specification	32
Appendix D - Web Application Report.....	33
Appendix E - Mobile Application Report	44

1. Introduction and Objectives

This report builds three deployable intelligent systems from raw Kaggle data - a **diabetes screening classifier**, a **house-price regressor**, and an **e-commerce product-recommendation classifier that also uses review text** - and takes each through the full pipeline, ending in a FastAPI POST /predict service consumed by a React/Vite web page and a Flutter mobile app. The purpose is to demonstrate how three different kinds of real-world data become numerical representations that a machine-learning model can process.

Application	Task	Main representation
Diabetes	Binary classification - diabetic vs not	scaled + passthrough feature matrix X in $\mathbb{R}^{N \times 23}$
House price	Regression - sale price (log1p, million VND)	imputed + scaled + one-hot matrix X in $\mathbb{R}^{N \times 92}$
Customer behaviour	Binary classification - will the reviewer recommend the product	tabular one-hot+scaled matrix X in $\mathbb{R}^{N \times 138}$ + TF-IDF text vectors $\mathbb{R}^{\sim 28\ 600}$

Field	Diabetes	House Price	Customer Behaviour
Dataset	Diabetes Health Indicators (BRFSS 2015)	VN Real Estate Listings (Apr-Sept 2025)	Sephora Products and Skincare Reviews (nadyinky)
Kaggle	kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset	kaggle.com/datasets/qmanhbeo/vietnamese-real-estate-listings-may-2024 (CC BY-NC 4.0; Guland.vn scrape)	kaggle.com/datasets/nadyinky/sephora-products-and-skincare-reviews
Rows (raw -> clean)	253 680 -> 229 781	236 226 -> 201 654	116 262 -> 104 313
Target	Diabetes_binary in $\{0,1\}$ (17.3% positive)	log1p(Price) in million VND	is_recommended in $\{0,1\}$ (84.7% positive)
Selected model	Random Forest	Random Forest	Logistic Regression on tabular + TF-IDF text
Headline test metric	ROC-AUC 0.809 , recall(1) 0.735 , F1 0.48, acc 0.725	RMSE 26 873 (million VND), R^2 0.186 , MAE 10 626	ROC-AUC 0.964 , macro-F1 0.858 , recall(0) 0.876 , acc 0.918
Deployment	FastAPI + React/Vite web + Flutter	FastAPI + React/Vite web + Flutter	FastAPI + React/Vite web + Flutter

2. Connection to Lecture 02 - Data Representation

Real-world object -> Raw data -> Numerical representation -> Tensor -> Model
(patient / house / review) (CSV row) (feature vector x) (X matrix / + E) (R
 F / RF / $LogReg$)

- A **tabular sample** is a vector $x = [x_1, \dots, x_d]^T$ in R^d ; a dataset is a matrix X in $R^{N \times d}$ (N rows = samples, d columns = features after encoding).
- **Categorical** -> **numeric** by one-hot encoding, e.g. Property Type in {Nhà riêng, Căn hộ, ...} -> Nhà riêng = $[1, 0, 0, \dots]$. Binary survey answers are already $\{0, 1\}$; ordinal survey codes (GenHlth 1-5, Age bracket 1-13) are kept as monotonic numbers.
- **Numeric** -> **scaled** with StandardScaler on the continuous columns; skewed money columns get $\log_{10}$ first.
- **Text** -> **numbers** by Text -> Tokens -> Token IDs -> Embeddings. For App 3 a batch of embedded reviews is a 3-D tensor E in $R^{B \times T \times d}$ (B reviews, T tokens each, d numbers per token); the deployed model instead uses the 2-D sparse **TF-IDF** form.
- **The representation is part of the solution** - marks are for explaining *how raw data becomes computational data*, not only for accuracy.

3. Data Sources and Application Definitions

For each application, one row of the raw data and the target are defined below; the full dataset tables (name, URL, observations, attributes, feature names, data types) are in Sections 4.2, 5.2 and 6.2.

Question	Diabetes	House price	Customer behaviour
What does one row represent?	one BRFSS phone respondent	one property listing	one product review
Raw data representation	CSV, 22 numeric columns	CSV, ~30 mixed columns + free text	2 joined CSVs, mixed columns + review_title/review_text
Input feature columns	21 raw survey items + 2 engineered	11 numeric + 7 categorical (post-engineering)	15 numeric/binary + 6 categorical + 1 text
Target column	Diabetes_binary in $\{0, 1\}$	Price (modelled as $\log_{10}$)	is_recommended in $\{0, 1\}$
Which features need encoding?	none (binary/ordinal already numeric)	Property Type / Position / Direction / Road Type / Province / Agent Role / district -> one-hot	skin_type / skin_tone / eye_color / hair_color / secondary_category / brand_name -> one-hot; review_text -> TF-IDF
Which features need normalization?	8 continuous/count columns ->	numeric block -> $\log_{10}$ (skewed) then	9 numeric columns -> $\log_{10}$ (money/popularity)

Question	Diabetes	House price	Customer behaviour
	StandardScaler	StandardScaler	then StandardScaler
Final feature dimension d	23	92	28 733 (sparse)
Model input shape	$X \in \mathbb{R}^{229\,781 \times 23}$; one request $\mathbb{R}^{1 \times 23}$	$X \in \mathbb{R}^{201\,654 \times 92}$; one request $\mathbb{R}^{1 \times 92}$	$X \in \mathbb{R}^{104\,313 \times 28\,733}$; one request $\mathbb{R}^{1 \times 28\,733}$

Mandatory Data-Representation Summary

Application	Raw form	Numerical representation	Model input (B x d, or B x T x d)
Diabetes	CSV / table	scaled + passthrough feature matrix	N x 23 (N = 229 781); one request 1 x 23
House price	CSV / table	median-impute -> log1p -> scale (numeric) + one-hot (categorical)	N x 92 sparse (N = 201 654); target y in $\mathbb{R}^H = \log(1 + \text{Price})$
E-commerce	CSV + review comments	tabular one-hot + scaled, concatenated with a TF-IDF (1-2 gram) of the review text	N x 28 733 sparse (N = 104 313); tokenisation demo B x T x d = 1 x 40 x 16

Every dimension. N = cleaned rows (after the Section 4/5/6 filtering). d for diabetes = 8 scaled + 15 passthrough. d for house price = 11 numeric + 81 one-hot columns after min_frequency = 50. d for e-commerce = 138 tabular columns after one-hot, plus 28 595 TF-IDF features on the training vocabulary. In the embedding demo B = 1 review, T = 40 tokens after padding/truncation, d = 16 embedding width.

4. Application 1 - Diabetes Prediction

4.1 Problem description

Predict whether a survey respondent **has diabetes** from routine health-behaviour and demographic items, so a primary-care service can prioritise high-risk people for a confirmatory blood test. X = 21 BRFSS survey items (HighBP, HighChol, BMI, GenHlth, Age, ...); y = Diabetes_binary in {0, 1}, derived from the 3-class Diabetes_012 as ≥ 1 . **Binary classification;** one observation = one respondent.

4.2 Dataset

Field	Value
Name / source	Diabetes Health Indicators (CDC BRFSS 2015, Kaggle alexteboul/diabetes-health-indicators-dataset)

Field	Value
Observations	253 680 raw -> 229 781 after de-duplication
Attributes	21 features + target; all numeric float64 (14 binary, 4 ordinal codes, 3 continuous)
Target	Diabetes_binary - positive rate 15.76 % raw -> 17.29 % after de-duplication
One observation	one respondent's answers, e.g. HighBP=1, HighChol=1, BMI=40, GenHlth=5, Age=9, DiffWalk=1, ... -> Diabetes_binary=1

4.3 Data understanding and quality

df.shape = (253 680, 22); df.info() - all columns numeric (float64); df.isna().sum() - **0 missing values**; df.duplicated().sum() - **23 899 exact duplicate rows**; df.describe() - BMI ranges 12-98, MentHlth/PhysHlth are mostly 0 (genuine survey answers). There are **no categorical or text columns** and no invalid values. The target is **imbalanced** (17.3 % positive). The executed inspection/quality cells are in diabetes/notebook/diabetes.ipynb (Sections 4-9).

4.4 Data cleaning

Operation	What	Why
Drop exact duplicates	23 899 rows removed before the split	an identical answer vector in both train and test is optimistic leakage
Keep BMI outliers	no clip / drop	the Section 9 sensitivity check (clip to [12,80] / [14,60] / drop) moves ROC-AUC by ≤ 0.003
Median imputer inside the pipeline	fitted on train only	robustness to a field missing at inference, one code path
Ordinal codes kept as numbers	GenHlth / Age / Education / Income	monotonic with risk; one-hot would add ~30 sparse columns for no gain

4.5 Data representation

$$X_{\{diabetes\}} \text{ in } R^{229781 \times 23}, y \text{ in } \{0,1\}^{229781} \quad (\text{Eq. 4.1})$$

```
# Diabetes ColumnTransformer Pipeline Implementation
preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler())
    ]), ['BMI', 'MentHlth', 'PhysHlth', 'TotalUnhealthyDays', ...]),
    ('bin', 'passthrough', ['HighBP', 'HighChol', 'Stroke', 'CardioRisk', ...])
])
```

One row record -> engineer() adds TotalUnhealthyDays = clip(MentHlth + PhysHlth, 0, 60) and CardioRisk = Stroke OR HeartDiseaseorAttack -> **23 model features** (8 continuous/count -> StandardScaler, 15 binary/ordinal passed through). Example:

raw : HighBP=1, HighChol=1, BMI=40.0, GenHlth=5, Age=9, DiffWalk=1, ...
vector : $x = [x_1, \dots, x_{23}]^T$ in $\mathbb{R}^{23}$ (8 scaled values, 15 passthrough 0/1/ordinal)
matrix : X in $\mathbb{R}^{229\ 781 \times 23}$, dense float64; y in $\{0,1\}^{229\ 781}$
request: one API call = $\mathbb{R}^{1 \times 23}$

12. Data representation

This section makes the link to Lecture 02 explicit. A tabular sample is a vector $x = [x_1, \dots, x_d]^T \in \mathbb{R}^d$ and the dataset is a matrix $X \in \mathbb{R}^{N \times d}$. The chain from file to model input is

CSV -> DataFrame -> clean feature matrix -> scaled matrix -> model input

The cell below prints one raw record together with the feature vector it becomes, and reports the original DataFrame shape, the final X shape and its dtype. These are the figures used in the report's data-representation summary.

```
In [12]: from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

def engineer(frame: pd.DataFrame) -> pd.DataFrame:
    # Deterministic, stateless feature step - applied identically in training and inference.
    # No BMI clipping (decision in section 9) - the only cleaning was the section 7 duplicate drop.
    out = frame.copy()
    out["TotalUnhealthyDays"] = (out["MentHlth"] + out["PhysHlth"]).clip(0, 60)
    out["CardioRisk"] = ((out["Stroke"] == 1) |
                        (out["HeartDiseaseorAttack"] == 1)).astype(int)
    return out

MODEL_FEATURES = RAW_FEATURES + ["TotalUnhealthyDays", "CardioRisk"] # d = 23
SCALE_FEATURES = CONTINUOUS_FEATURES + ORDINAL_FEATURES + ["TotalUnhealthyDays"] # 8
PASS_FEATURES = BINARY_FEATURES + ["CardioRisk"] # 15

raw_row = df.iloc[0][RAW_FEATURES]
print("one RAW csv record (21 features):")
print(raw_row.to_dict())

demo_X = engineer(df)[MODEL_FEATURES]
demo_pre = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                    ("scale", StandardScaler())])), SCALE_FEATURES),
    ("bin", "passthrough", PASS_FEATURES),
])
demo_mat = demo_pre.fit_transform(demo_X) # demo only - real fit is on the train split (section 15)

print("nsame record as a FEATURE VECTOR x (8 scaled + 15 passthrough), x in R^23:")
print(np.round(demo_mat[0], 3))
print("\noriginal DataFrame shape :", df.shape,
      "\n# (N rows, 23 cols = Diabetes_012 + 21 raw features + derived Diabetes_binary)")
print("feature matrix X shape :", demo_mat.shape, "\n# X in R^{N x d}, d = 23")
print("target vector y shape :", df["Diabetes_binary"].shape, "\n# y in {0,1}^N")
print("dtype of X :", demo_mat.dtype, "\n# dense float64 ndarray")
print("one API request is : R^{1 x 23} # a single feature vector")

one RAW csv record (21 features):
{'HighBP': 1.0, 'HighChol': 1.0, 'CholCheck': 1.0, 'Smoker': 1.0, 'Stroke': 0.0, 'HeartDiseaseorAttack': 0.0, 'PhysActivity': 0.0, 'Fruits': 0.0, 'Veggies': 1.0, 'HvyAlcoholConsump': 0.0, 'AnyHealthcare': 1.0, 'NoDocbcCost': 0.0, 'DiffWalk': 1.0, 'Sex': 0.0, 'GenHlth': 5.0, 'Age': 9.0, 'Education': 4.0, 'Income': 3.0, 'BMI': 40.0, 'MentHlth': 18.0, 'PhysHlth': 15.0}

same record as a FEATURE VECTOR x (8 scaled + 15 passthrough), x in R^23:
[ 1.667  1.879  1.141  2.253  0.295 -0.988 -1.381  1.806  1.  1.
  1.  1.  0.  0.  0.  0.  1.  0.  1.  0.
  1.  0.  0. ]

original DataFrame shape : (229781, 23) # (N rows, 23 cols = Diabetes_012 + 21 raw features + derived Diabetes_binary)
feature matrix X shape : (229781, 23) # X in R^{N x d}, d = 23
target vector y shape : (229781,) # y in {0,1}^N
dtype of X : float64 # dense float64 ndarray
one API request is : R^{1 x 23} # a single feature vector
```

Figure 4.1

Figure 4.1 - Diabetes: one raw record -> its 23-dim feature vector -> X shape / dtype (notebook Section 12).

4.6 Exploratory data analysis

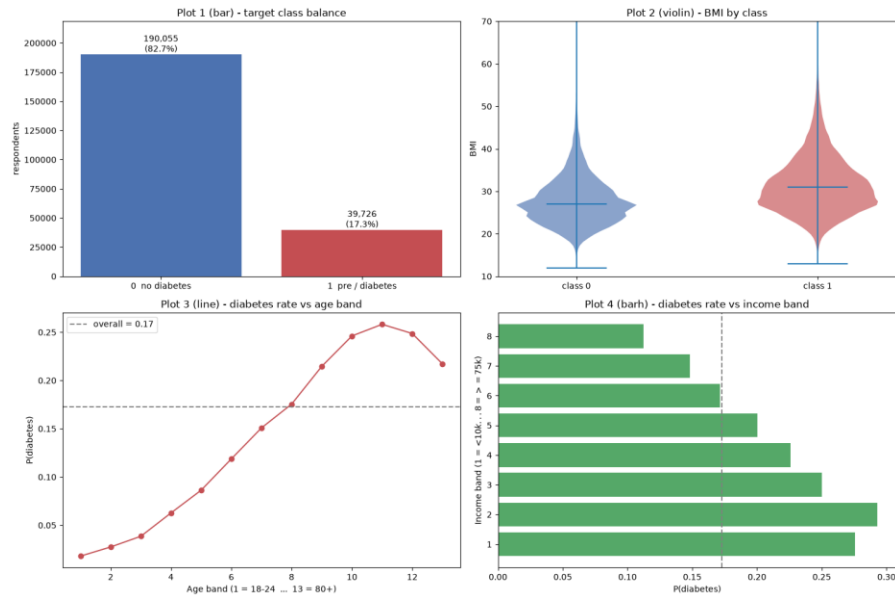


Figure 4.2

Figure 4.2 - Diabetes: EDA plot grid - target balance, GenHlth vs diabetes rate, BMI distribution by class, Age bracket vs rate.

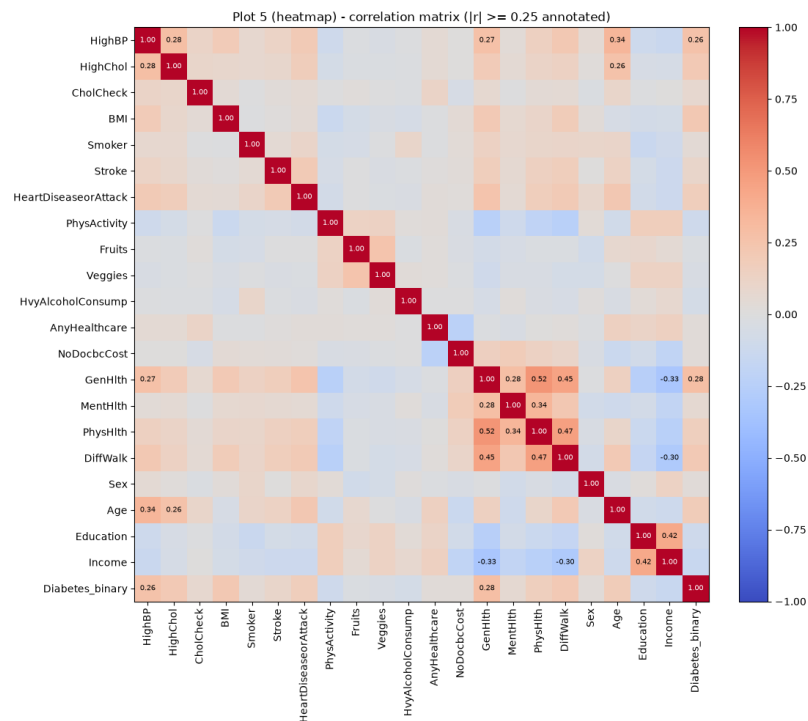


Figure 4.3

Figure 4.3 - Diabetes: feature / target correlation heatmap.

- **Target balance** - 17.3 % positive. *Observation:* a majority classifier already scores 0.83 accuracy. *ML implication:* judge on recall / F1 / ROC-AUC, not accuracy.
- **GenHlth vs diabetes rate** - monotonic, ~4 % at “excellent” to ~35 % at “poor”.
Interpretation: self-rated health is the single strongest predictor. *ML implication:* keep GenHlth as an ordinal number.
- **BMI distribution by class** - diabetic respondents ~4 BMI points higher, large overlap.
Interpretation: BMI helps but does not separate the classes alone.
- **Age bracket vs rate** - rises steadily with age. *ML implication:* keep Age ordinal.
- **Correlation heatmap (Figure 4.3)** - HighBP, HighChol, GenHlth, DiffWalk, Age, BMI are most correlated with the target; lifestyle items (Fruits, Veggies) near zero. *ML implication:* all columns kept (a tree ensemble is robust to weak features); no single feature dominates.

4.7 Model development

Five models on the same 25 000-row stratified training subsample and the same preprocessing; the winner is refitted on the full training split.

Model	Hyperparameters
Logistic Regression	class_weight="balanced", max_iter=1000
Decision Tree	max_depth=6, min_samples_leaf=50, class_weight="balanced"
Random Forest	n_estimators=300, max_depth=12, min_samples_leaf=20, class_weight="balanced"
SVM (RBF)	class_weight="balanced"
KNN	default n_neighbors

Split: stratified 70/15/15 (train_test_split twice, seed 42) -> train 160 846 / val 34 467 / test 34 468, each at positive rate 0.1729. Every fitted object is fitted on X_train only - the test set never influences training or preprocessing.

4.8 Evaluation and model comparison

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN), \quad \text{Precision} = TP / (TP + FP) \quad (\text{Eq. 4.2})$$

$$\text{Recall} = TP / (TP + FN), \quad F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (\text{Eq. 4.3})$$

$$\text{Loss}_{\{BCE\}} = - (1/N) * \sum_i [y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i)] \quad (\text{Eq. 4.4})$$

18. Model comparison

```
In [18]: rows = []
for name, pipe in fitted.items():
    s = score(pipe, X_val, y_val)
    rows.append([name, s["accuracy"], s["precision"], s["recall"], s["f1"], s["roc_auc"]])
cmp = (pd.DataFrame(rows, columns=["model", "accuracy", "precision", "recall", "f1", "roc_auc"])
      .round(3).sort_values("roc_auc", ascending=False).reset_index(drop=True))
print("baseline B (3-feature LogReg) ROC-AUC for reference:",
      round(score(base_cheap, X_val[CHAP], y_val)["roc_auc"], 3))
cmp
```

baseline B (3-feature LogReg) ROC-AUC for reference: 0.77

```
Out[18]:
```

	model	accuracy	precision	recall	f1	roc_auc
0	RandomForest	0.727	0.361	0.748	0.487	0.810
1	LogisticRegression	0.716	0.352	0.760	0.481	0.804
2	SVM_RBF	0.707	0.345	0.776	0.478	0.800
3	KNN	0.830	0.550	0.104	0.174	0.790
4	DecisionTree	0.690	0.328	0.760	0.459	0.787

```
In [19]: # stability check: does the winner's quality survive going from 25k -> full training data?
BEST_NAME = cmp.iloc[0]["model"]
best_full = Pipeline([("prep", make_preprocessor()),
                      ("clf", sklearn.clone(model_specs[BEST_NAME]))]).fit(X_train, y_train)
stab = pd.DataFrame({
    f"{BEST_NAME} @ 25k subsample": score(fitted[BEST_NAME], X_val, y_val),
    f"{BEST_NAME} @ full train": score(best_full, X_val, y_val),
}).T.round(3)
print("model carried to the held-out test set:", BEST_NAME)
stab
```

model carried to the held-out test set: RandomForest

```
Out[19]:
```

	accuracy	precision	recall	f1	roc_auc
RandomForest @ 25k subsample	0.727	0.361	0.748	0.487	0.810
RandomForest @ full train	0.723	0.358	0.760	0.486	0.812

Figure 4.4

Figure 4.4 - Diabetes: five-model comparison table and the stability check (notebook Section 18).

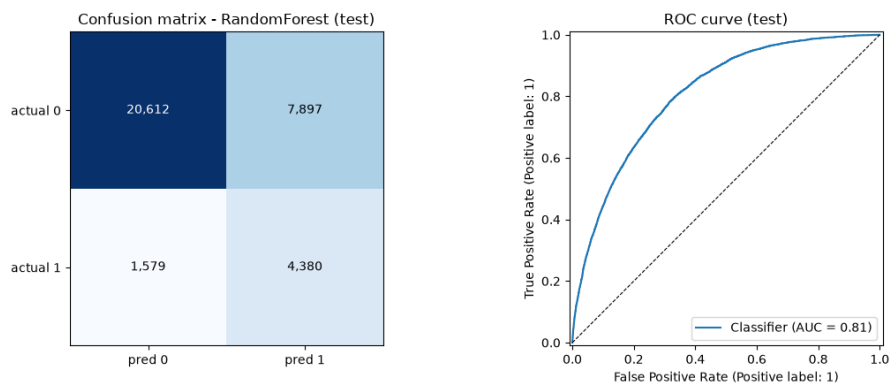


Figure 4.5

Figure 4.5 - Diabetes: classification report, confusion-matrix heatmap and ROC curve for the held-out test (notebook Section 19).

Validation (25 000-row fit):

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	0.727	0.361	0.748	0.487	0.810

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.716	0.352	0.760	0.481	0.804
SVM (RBF)	0.707	0.345	0.776	0.478	0.800
KNN	0.830	0.550	0.104	0.174	0.790
Decision Tree	0.690	0.328	0.760	0.459	0.787
baseline B (3-feature LogReg)	0.699	0.328	0.708	0.448	0.77
baseline A (majority class)	0.827	0.000	0.000	0.000	0.50

Held-out test - Random Forest, refit on full train: accuracy 0.725, ROC-AUC **0.809**, recall on the diabetic class **0.735** (precision 0.357, F1 0.480). **Confusion matrix** **[[20 612, 7 897], [1 579, 4 380]]**: the 1 579 false negatives are diabetic people told they are fine - the dangerous error; the 7 897 false positives are healthy people sent for a blood test - a cost, not a harm. **Most important metric: recall on class 1.** A missed diabetic is far worse than a wasted test, so the class-weighted model trades precision for recall on purpose, and overall accuracy (0.725) sits below the 0.83 majority baseline.

4.9 Model selection

Random Forest - highest ROC-AUC, best recall/F1 balance, clears both baselines, feature_importances_ for interpretation, ~2 s to train, one small artifact. Logistic Regression is the documented lightweight fallback (near-identical ROC-AUC, coefficient-level interpretability).

4.10 Persistence and deployment

Persisted as diabetes/model/model_pipeline.joblib = Pipeline([prep, RandomForestClassifier]), plus feature_names.joblib and input_schema.json. The Section 23 inference test reloads the file and confirms predict_proba matches the in-memory model to 10 decimals; example raw input -> {"prediction": "diabetic", "confidence": 0.8496}. Deployment: Section 8, Appendix D.1 (web), Appendix E (mobile).

5. Application 2 - House-Price Prediction

5.1 Problem description

Predict a fair listing price from property attributes so a marketplace can flag mispriced listings. X = property + location + agent attributes; y = Price in million VND, modelled as $\log(1 + \text{Price})$ (raw skew 3.36 -> -0.07) and inverted with $\exp m1$. Regression; one observation = one listing. It differs from Application 1 because the target is a continuous positive quantity, so the metrics are MAE / MSE / RMSE / R^2 , not precision/recall.

5.2 Dataset

Field	Value
Name / URL	Vietnamese Real Estate Listings 2025 (Kaggle qmanhbeo/vietnamese-real-estate-listings-may-2024)

Field	Value
Licence / provenance	CC BY-NC 4.0 ; scraped from public sale adverts on Guland.vn, Scraped At = 12-14 Sept 2025; file ~210 MB, not committed
Observations	236 226 raw -> 201 654 after cleaning
Numeric features	Area, Width, Length, Bedrooms, Bathrooms, Floors, Alley Width, Agent Listing Count + 3 *_missing indicators
Categorical features	Property Type, Position, Direction, Road Type, Province, Agent Role, district
Target	Price (million VND) -> log1p
One observation	one advert, e.g. Area=192, Property Type="house", Location="Long Xuyen, An Giang", ... -> Price=2000 (million VND)

5.3 Data understanding and cleaning

df.shape = (236 226, ~30); df.info() separates numeric columns from categorical/text (the executed cells are in house_price/notebook/house_price.ipynb). df.isna().sum() shows **structural missingness** - Bedrooms 72 %, Bathrooms 84 %, Floors 79 %, Length 58 %, Latitude/Longitude 68 %. Invalid values: 611 non-positive and ~10 600 implausible (> 200 000 million VND) prices, plus out-of-range areas.

Operation	What	Why
Drop impossible rows	Price <= 0 or > 200 000; Area <= 0 or > 10 000	not real values; would dominate the loss
*_missing indicator features	Bedrooms_missing, Bathrooms_missing, Floors_missing	the fact that a field is blank is itself informative
Median-impute the rest, inside the pipeline	fitted on train only	one training/inference path, no leakage
Parse district from Location	one-hot with min_frequency = 50	province + district is the main price driver ($\eta^2 \sim 0.24$)
log(1 + Price) target		raw skew 3.36; the QQ-plot is straight after log1p
Drop Listing ID, VIP Account (constant), Title		ids leak nothing; a constant column is dead; the title literally contains the price string

5.4 Representation

$$y_i' = \log(1 + y_i), \quad \hat{y}_i = \exp(\hat{y}_i') - 1 \quad (\text{Eq. 5.1})$$

$$X_{\text{house}} \text{ in } R^{\{201654 \times 92\}}, \quad y \text{ in } R^{\{201654\}} \text{ (million VND)} \quad (\text{Eq. 5.2})$$

```
# House Price Preprocessing Pipeline Implementation
preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('log1p', FunctionTransformer(np.log1p)),
        ('scale', StandardScaler())
    ]), numeric_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', min_frequency=50),
 categorical_cols)
])
```

Cleaned frame (201 654, 31) -> ColumnTransformer: SimpleImputer(median) -> log1p -> StandardScaler (numeric); OneHotEncoder(handle_unknown="ignore", min_frequency=50) (7 categoricals) -> X in $\mathbb{R}^{\{201\ 654 \times 92\}}$ (sparse); y in $\mathbb{R}^H = \log(1 + \text{Price})$. One API request -> $\mathbb{R}^{\{1 \times 92\}}$; the response inverts with $\exp m1$.

12. Data representation

CSV -> DataFrame -> clean feature frame -> ColumnTransformer (impute + scale + one-hot) -> X -> model

The cell below prints one raw listing and the feature vector it becomes, together with the DataFrame shape, the final X shape and its dtype.

```
In [28]: from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# minimal engineered frame just for this demo (full version in section 13)
NUM_DEMO = ['Area', 'Width', 'Length', 'Bedrooms', 'Bathrooms', 'Floors', 'Alley Width']
CAT_DEMO = ['Property Type', 'Position', 'Direction', 'Road Type', 'Province', 'Agent Role']

demo_df = df[NUM_DEMO + CAT_DEMO].copy()
for c in CAT_DEMO:
    demo_df[c] = demo_df[c].fillna('Unknown')
y_demo = np.log1p(df['Price'].values) # model target = log1p(Price)

demo_prep = ColumnTransformer([
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler())
    ]), NUM_DEMO),
    ('cat', OneHotEncoder(handle_unknown='ignore', min_frequency=20), CAT_DEMO),
])
X_demo = demo_prep.fit_transform(demo_df)

print('one RAW listing (selected columns):')
print(df.iloc[0][['Title', 'Price', 'Area', 'Property Type', 'Location']].to_dict())
print('\nsame listing as a FEATURE VECTOR x (first 15 dims of the imputed+scaled+one-hot row):')
row0 = X_demo[0].toarray().ravel() if hasattr(X_demo, 'toarray') else X_demo[0]
print(np.round(row0[:15], 3))
print('\nDataFrame shape at this point:', df.shape, ' (after sections 7-8 cleaning)')
print('feature matrix X shape      : ', X_demo.shape, ' # X in  $\mathbb{R}^{\{N \times d\}}$ ')
print('target vector y shape      : ', y_demo.shape, ' # y in  $\mathbb{R}^N$  (log1p million VND)')
print('dtype of X                  : ', X_demo.dtype)
print('one API request is          :  $\mathbb{R}^{\{1 \times d\}}$  (a single listing)')

one RAW listing (selected columns):
{'Title': 'Bán nhà 192m² 2 tỷ tại Phường Mỹ Hòa Thành phố Long Xuyên phía sau nhà có thêm mảnh vườn nhỏ tầm 150m²', 'Price': 2000.0, 'Area': 192.0, 'Property Type': 'Nhà riêng', 'Location': 'Phường Long Xuyên, An Giang(Mới)'}

same listing as a FEATURE VECTOR x (first 15 dims of the imputed+scaled+one-hot row):
[-0.223 -0.005 -0.002 -0.007 -0.133 -0.002 -0.002  0.      0.
  1.      0.      0.      0.      0.      0.      0. ]

DataFrame shape at this point : (201654, 31) (after sections 7-8 cleaning)
feature matrix X shape      : (201654, 92) # X in  $\mathbb{R}^{\{N \times d\}}$ 
target vector y shape      : (201654,) # y in  $\mathbb{R}^N$  (log1p million VND)
dtype of X                  : float64
one API request is          :  $\mathbb{R}^{\{1 \times d\}}$  (a single listing)
```

Figure 5.1

Figure 5.1 - House price: one raw listing -> its 92-dim feature row -> X / y shape (notebook Section 12).

5.5 Exploratory data analysis

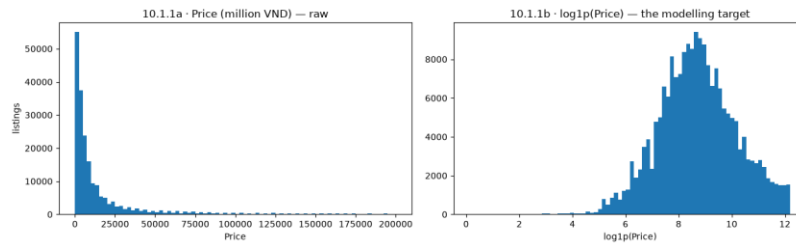


Figure 5.2

Figure 5.2 - House price: target distribution - Price raw vs $\log(1 + \text{Price})$.

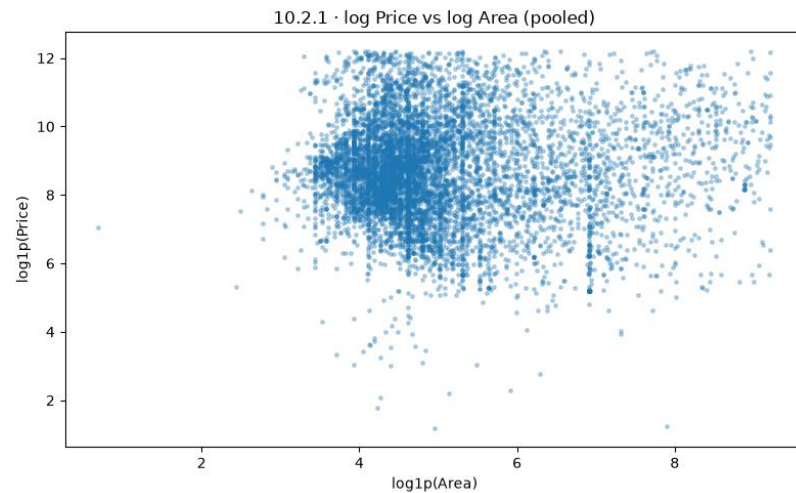


Figure 5.3

Figure 5.3 - House price: numeric correlation heatmap.

- Distributions (10.1) - Price raw skew 3.36 $\rightarrow$ log1p -0.07; Area skew 5.11 $\rightarrow$ 1.32. ML implication: train on $\log(1 + \text{Price})$, report metrics back on the million-VND scale.
- Relationships and correlation (10.2, Figure 5.3) - surprisingly weak numeric signal: $|\text{r}(\log \text{Area}, \log \text{Price})| \sim 0.02$, Bathrooms 0.215 is the strongest numeric; categorical correlation ratios Province $\eta^2 \sim 0.24$, Property Type ~ 0.12 . Interpretation: location and property type carry most of the (limited) signal; raw area is unreliable because listings mix land and built-up property. ML implication: this is a weak-signal regression problem - a tree ensemble captures the location x type structure, and the low R^2 is reported honestly as a data limitation.
- **Supporting analyses (10.3)** - missingness by property type justifies the *_missing indicators; the skew/QQ view justifies the log1p target.

5.6 Regression models and evaluation

$$MAE = (1/N) * \sum_i |y_i - \hat{y}_i|, \quad RMSE = \sqrt{(1/N) * \sum_i (y_i - \hat{y}_i)^2} \quad (\text{Eq. 5.3})$$

$$R^2 = 1 - [\sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2] \quad (\text{Eq. 5.4})$$

18. Model comparison

Validation MAE / MSE / RMSE / R^2 / MAPE and train time for all five models, ranked by RMSE.

```
In [34]: rows = []
for name, pipe in fitted.items():
    s = regression_scores(y_val, pipe.predict(X_val))
    s['model'] = name; s['train_s'] = round(timings[name], 2)
    rows.append(s)
cmp = pd.DataFrame(rows).set_index('model')[['MAE', 'MSE', 'RMSE', 'R2', 'MAPE_%', 'train_s']]
cmp = cmp.sort_values('RMSE')
cmp.round(3)
```

```
Out[34]:
```

model	MAE	MSE	RMSE	R2	MAPE_%	train_s
RandomForest	10810.710	7.479634e+08	27348.919	0.183	311.735	8.53
XGBoost	10782.482	7.531540e+08	27443.652	0.178	315.436	1.19
DecisionTree	11522.087	7.835407e+08	27991.797	0.144	349.132	0.74
LinearRegression	12149.187	8.898551e+08	29830.439	0.028	375.281	0.67
Ridge	12152.914	8.916251e+08	29860.092	0.026	375.585	0.55

Figure 5.4

Figure 5.4 - House price: five-model regression comparison table (notebook Section 18).

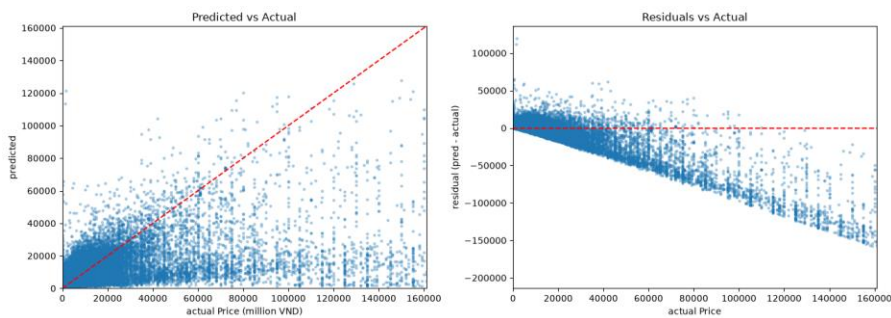


Figure 5.5

Figure 5.5 - House price: predicted-vs-actual scatter and residuals-vs-actual for the Random Forest (notebook Section 19).

Five models, 50 000-row training subsample, same preprocessing:

Model	val MAE	val RMSE	val R^2
Random Forest	10 811	27 349	0.183
XGBoost	10 794	27 478	0.176
Decision Tree (max_depth=12, min_samples_leaf=25)	11 522	27 992	0.144
Linear Regression	12 149	29 830	0.028
Ridge	12 153	29 860	0.026
baseline (predict the median)	14 397	32 185	-0.131

Held-out test - Random Forest, refit on full train: MAE 10 626, MSE 722 149 435, RMSE 26 873, R^2 0.186, MAPE 350 % (all money in million VND). What each metric means here: MAE - the average listing is off by about 10.6 billion VND, large because the price range spans three orders of magnitude; RMSE is $\sim 2.5\times$ the MAE, so a minority of listings are missed by far more than the typical one; R^2 0.186 - the model explains $\sim 19\%$ of the variance in $\log(1 + \text{Price})$ beyond a constant-mean predictor (which scores $R^2 = -0.13$), so it adds real signal but most

price variance is not recoverable from these fields; MAPE is inflated by the many low-priced listings and is reported, not used to rank models. There is no confusion matrix (regression). The honest headline is a weak-signal regression problem, driven by the near-zero numeric correlation, the 70-84 % missing size fields and 68 % missing geolocation - a property of the dataset, not the algorithm.

5.7 Model selection, persistence and deployment

Random Forest - lowest test RMSE, highest R^2 , feature_importances_, ~15 s to train, bagging absorbs the price outliers. Persisted as house_price/model/model_pipeline.joblib (Pipeline([prep, RandomForestRegressor])) + feature_names.joblib + input_schema.json (records price_unit = million VND and the log1p/expm1 convention). Section 23 inference test: disk == in-memory. Deployment: Section 8, Appendix D.2, Appendix E.

6. Application 3 - E-Commerce Customer Behaviour and Interest

6.1 Problem description

Predict whether a Sephora skincare reviewer **recommends the product**, from **who they are** and **what they wrote**. Target is _recommended in {0, 1} - a field *separate* from the 1-5 star rating (the rating is excluded as leakage, Section 6.6). **Binary classification**; one observation = one product review. Supports review-consistency QA, cold-start ranking, and merchandising.

6.2 Dataset

Field	Value
Name / URL	Sephora Products and Skincare Reviews (Kaggle nadyinky/sephora-products-and-skincare-reviews, CC0)
Files	reviews_500-750.csv joined to product_info.csv on product_id
Observations	116 262 raw -> 104 313 after cleaning; positive rate 0.8465
Scope	249 products / ~79 brands / 12 secondary_category values, all Skincare
Feature types	5 numeric log1p+scale (price_usd, loves_count, reviews, total_feedback_count, total_neg_feedback_count), 4 numeric scale (n_ingredients, n_highlights, review_age_days, pos_feedback_ratio), 6 binary passthrough, 6 one-hot categorical (skin_type, skin_tone, eye_color, hair_color, secondary_category, brand_name), 1 text (review_title + " ." + review_text)
Excluded (leakage)	the review's own rating and the product average rating_product

Field	Value
One observation	one review: a skin profile + product fields + a title and body of free text -> is_recommended in {0,1}

df.shape = (116 262, ~30); df.isna().sum() - 11 803 rows with a blank target, many blank skin-profile fields; df.duplicated().sum() - a few exact duplicates; 125 empty bodies; class balance **84.7 % positive**. The executed cells are in customer_behaviour/notebook/customer_behaviour.ipynb (Sections 4-9).

6.3 Customer representation

$X_{\text{ecom}} = [X_{\text{tabular}} \parallel X_{\text{text}}]$ in $\mathbb{R}^{104313 \times 28733}$, E in $\mathbb{R}^{H \times T \times e}$ (Eq. 6.1)

```
# E-Commerce Tabular + TF-IDF Vectorizer Pipeline Implementation
preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')), ('scale',
StandardScaler())]), num_cols),
    ('cat', OneHotEncoder(min_frequency=25, handle_unknown='ignore'), cat_cols),
    ('txt', TfidfVectorizer(ngram_range=(1,2), min_df=10, max_features=40000,
stop_words='english'), 'review_text')
])
```

Tabular: one raw review -> 21 columns -> SimpleImputer(median) -> log1p (money / popularity) -> StandardScaler; binary passthrough; OneHotEncoder(handle_unknown= "ignore", min_frequency=25) -> **x_tab** in $\mathbb{R}^{138}$.

Text - the required Comment -> Tokens -> Token IDs -> Embedding demo on a real review:

Comment : "I will be the first to say that the price on these is a lot, but for what they do? ..."

Tokens : ['say', 'price', 'lot', 'unbeatable', 'deep', 'painful', 'nodules', ...] (T = 40)

Token IDs: [29, 26, 17, 39, 6, 22, 21, ...] (index into a 41-row vocab, 0 = PAD)

Embedding: each ID -> a row of a $(|V|+1) \times d$ table -> E in $\mathbb{R}^{T \times d} = \mathbb{R}^{40 \times 16}$

Batch : E_{batch} in $\mathbb{R}^{H \times T \times e} = \mathbb{R}^{1 \times 40 \times 16}$

The **deployed** model uses the sparse **TF-IDF** form (ngram_range=(1,2), min_df=10, max_features=40000, sublinear_tf, English stop-words) -> **x_txt** in $\mathbb{R}^{28\,595}$ on the training vocabulary. **Combined:** $X = [x_{\text{tab}} \parallel x_{\text{txt}}]$ in $\mathbb{R}^{104\,313 \times 28\,733}$ sparse.

12. Data representation

```
reviews_500-750 + product_info --join-> one review row
|
|-- build_features() -----> x_tab in R^d(d) (skin + product + timing; one-hot)
|-- TfIdfVectorizer(titlebody) -> x_txt (sparse) (word 1-2 grams)
y in (0,1)^N (is_recommended)

build_features() is stateless - a deterministic function of one review row, run identically in the notebook and the API. The cell also runs the required Comment -> Tokens -> Token IDs ->
Embedding demonstration (reporting B, T, d for E in R^{B x T x d}).

In [13]: from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfIdfVectorizer, CountVectorizer

def build_features(frame: pd.DataFrame) -> pd.DataFrame:
    '''Stateless review-row -> feature frame (tabular columns + the text column).
    Identical in the notebook and the API. Never touches the review's star rating or the
    product's average rating (leakage, 1.3 / 14a).'''
    d = frame.copy()
    def col(name, default=np.nan): # missing-column safe accessor
        return d[name] if name in d.columns else pd.Series(default, index=d.index)
    d['ts'] = pd.to_datetime(col("submission_time"), errors="coerce")
    ref = pd.Timestamp("2023-03-22") # fixed snapshot (max review date + 1)
    d['review_age_days'] = (ref - d['ts']).dt.total_seconds() / 86400
    for c in ["price_usd", "loves_count", "reviews"]:
        d[c] = pd.to_numeric(col(c), errors="coerce")
    d['price_missing'] = d['price_usd'].isna().astype(int)
    d['has_title'] = col("review_title", "").fillna("").astype(str).str.strip().ne("").astype(int)
    d['n_ingredients'] = col("ingredients", "").fillna("").astype(str).str.count(",") + 1
    d['n_highlights'] = col("highlights", "").fillna("").astype(str).str.count(",")
    for c in ["total_feedback_count", "total_pos_feedback_count", "total_neg_feedback_count"]:
        d[c] = pd.to_numeric(col(c), errors="coerce").fillna(0)
    d['pos_feedback_ratio'] = (d["total_pos_feedback_count"] / d["total_feedback_count"]).replace(0, np.nan).fillna(0.0)
    for c in ["limited_edition", "new", "online_only", "sephora_exclusive"]:
        d[c] = pd.to_numeric(col(c), errors="coerce").fillna(0).astype(int)
    for c in ["skin_type", "skin_tone", "eye_color", "hair_color", "secondary_category", "brand_name"]:
        d[c] = col(c).fillna("__na__").astype(str).str.strip().str.lower()
    d.loc[d.eye_color == "grey", "eye_color"] = "gray"
    d['review_all'] = (col("review_title", "").fillna("").astype(str) + " " +
                     + col("review_text", "").fillna("").astype(str)).str.strip()
    return d[DEPLOY_TAB + [TEXT_COL]]

df['n_ingredients'] = df.ingredients.fillna("").str.count(",") + 1
df['n_highlights'] = df.highlights.fillna("").str.count(",")
df['price_missing'] = df.price_usd.isna().astype(int)
df['has_title'] = df.review_title.fillna("").str.strip().ne("").astype(int)
for c in ["total_feedback_count", "total_pos_feedback_count", "total_neg_feedback_count"]:
    df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0)
df['pos_feedback_ratio'] = (df.total_pos_feedback_count / df.total_feedback_count.replace(0, np.nan)).fillna(0.0)

feat = build_features(df).reset_index(drop=True)
y_all = df["is_recommended"].astype(int).reset_index(drop=True)
print("one RMW review (selected):")
print(df.iloc[0][["author_id", "product_id", "skin_type", "skin_tone", "price_usd", "brand_name",
                  "secondary_category", "submission_time", "rating", "is_recommended"]].to_dict())
print(feat.iloc[0][DEPLOY_TAB].to_dict())
print("untxt field:", repr(feat.iloc[0][TEXT_COL][:160]))
print("unfeature frame:", feat.shape, "| y positive rate", round(y_all.mean(), 4))

one RMW review (selected):
{'author_id': '287984373', 'product_id': 'P442857', 'skin_type': 'normal', 'skin_tone': 'medium', 'price_usd': 20.0, 'brand_name': 'Dr. Jart+', 'secondary_category': 'treatment
s', 'submission_time': '2023-03-14', 'rating': 5, 'is_recommended': 1}

same review as the tabular feature row:
{'price_usd': 20.0, 'loves_count': 54955, 'reviews': 578.0, 'total_feedback_count': 0, 'total_neg_feedback_count': 0, 'n_ingredients': 0, 'n_highlights': 0, 'review_age_days': 8,
0, 'pos_feedback_ratio': 0.0, 'price_missing': 0, 'has_title': 1, 'limited_edition': 0, 'new': 0, 'online_only': 0, 'sephora_exclusive': 1, 'skin_type': 'normal', 'skin_tone': 'me
dium', 'eye_color': 'blue', 'hair_color': '__na__', 'secondary_category': 'treatments', 'brand_name': 'Dr. Jart+'}

text field: 'Best Pimple Patches . These are the only pimple patches I've used that actually work on hormonal acne.'

feature frame: (104313, 22) | y positive rate 0.8465

In [14]: # ----- required demo: Comment -> Tokens -> Token IDs -> Embedding -----
ex = df.review_text.dropna().iloc[7]
print("raw review :", repr(ex[:200]))
analyzer = CountVectorizer(strip_accents="unicode", stop_words="english").build_analyzer()
toks = analyzer.explode(ex)
print("tokens (T x)", len(toks), ")")
vocab = [w[1] for i, w in enumerate(sorted(set(toks)))] # 0 = PAD
ids = [vocab[w] for w in toks]
print("token IDs (T x)", len(ids))
T, d_emb, B = len(toks), 16, 1
E_table = np.random.default_rng(RANDOM_SEED).normal(0, 1, size=(len(vocab)+1, d_emb)).round(3)
E = E_table[ids]; E_batch = E[None, ...]
print(f"E (unembedding table) : {E_table.shape} ({V+1 rows, d={d_emb})")
print(f"E (one review) : {E.shape} -> T x d")
print(f"E_batch : {E_batch.shape} -> B x T x d (B={B}, T={T}, d={d_emb})")

tfidf_demo = TfIdfVectorizer(ngram_range=(1,2), min_df=10, max_features=40000,
                             sublinear_tf=True, strip_accents="unicode", stop_words="english")
Xtxt = tfidf_demo.fit_transform(feat[TEXT_COL])
print("TEXT representation used by the models: TF-IDF, Xtxt.shape, \"sparse CSR\"")

raw review : 'I will be the first to say that the price on these is a lot, but for what they do? They're unbeatable. I get deep and painful nodules thanks to stress/hormones/etc,
the type that left alone can be ain'
tokens (T = 40): ['sp', 'price', 'lot', 'unbeatable', 'deep', 'painful', 'nodules', 'thanks', 'stress', 'hormones', 'type', 'left', 'month', 'start', 'finish', 'patches', 'spee
d', 'healing', 'process', 'make', 'easy', 'care', 'spots', 'raise', 'module', 'surface', 'drain', 'help', 'prevent', 'scarring', 'followed', 'basic', 'hydrocolloid', 'patch', 'cut
s', 'healing', 'time', 'days', 'absolutely', 'insane']
token IDs : [29, 26, 17, 39, 6, 22, 21, 36, 34, 13, 38, 16, 19, 33, 9, 24, 31, 11, 27, 18, 8, 3, 32, 28, 20, 35, 7, 12, 25, 30, 10, 2, 14, 23, 4, 11, 37, 5, 1, 15]

embedding table : (40, 16) ({V+1 rows, d=16)
E (one review) : (40, 16) -> T x d
E_batch : (1, 40, 16) -> B x T x d (B=1, T=40, d=16)
TEXT representation used by the models: TF-IDF (104313, 40000) sparse CSR
```

Figure 6.1

Figure 6.1 - Customer behaviour: one raw review -> its tabular row and TF-IDF block, and the $B = 1$, $T = 40$, $d = 16$ embedding demo (notebook Section 12).

6.4 Data cleaning

Drop the 11 803 blank-target rows, 125 empty bodies and a few duplicates (116 262 -> 104 313). Skin-profile blanks -> an explicit __na__ one-hot level (missingness is a real behaviour). Incidental numeric gaps -> SimpleImputer(median) fitted **on train only**. Lower-case all categoricals; merge eye_color "grey" -> "gray". Outliers kept (log1p handles the skew). The TF-IDF vocabulary is built **on train only**. The review's own rating / rating_product are quarantined to the leakage demo.

6.5 Interest discovery / EDA

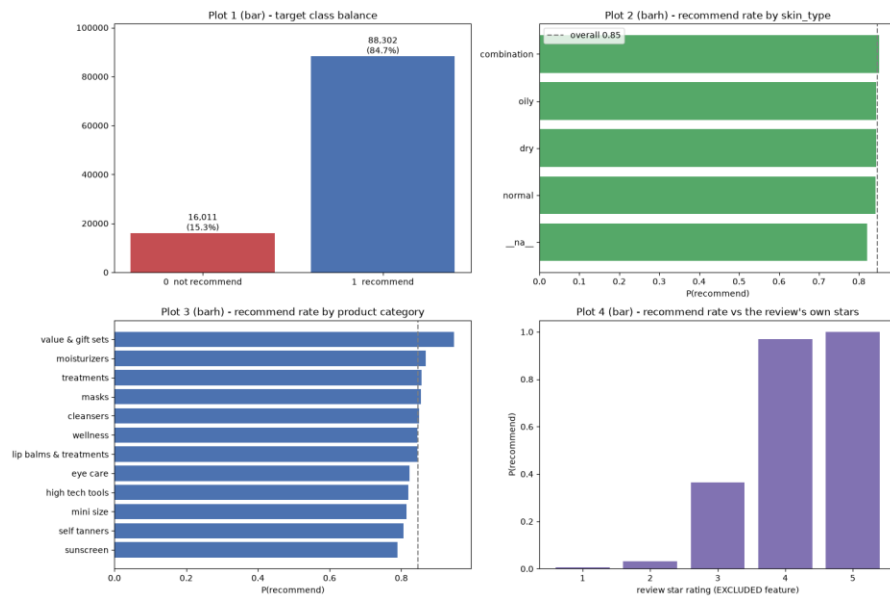


Figure 6.2

Figure 6.2 - Customer behaviour: EDA plot grid - class balance, recommend rate by skin type / category / brand / price decile, recommend rate vs the excluded star rating.

10. Exploratory data analysis

```
In [9]: CATS = ["skin_type","skin_tone","eye_color","hair_color","brand_name","secondary_category","tertiary_category"]
for c in CATS:
    df[c] = df[c].fillna("_na_").astype(str).str.strip().str.lower()
df.loc[df.eye_color.isin(["grey"]), "eye_color"] = "gray"
pos = df.is_recommended.mean()

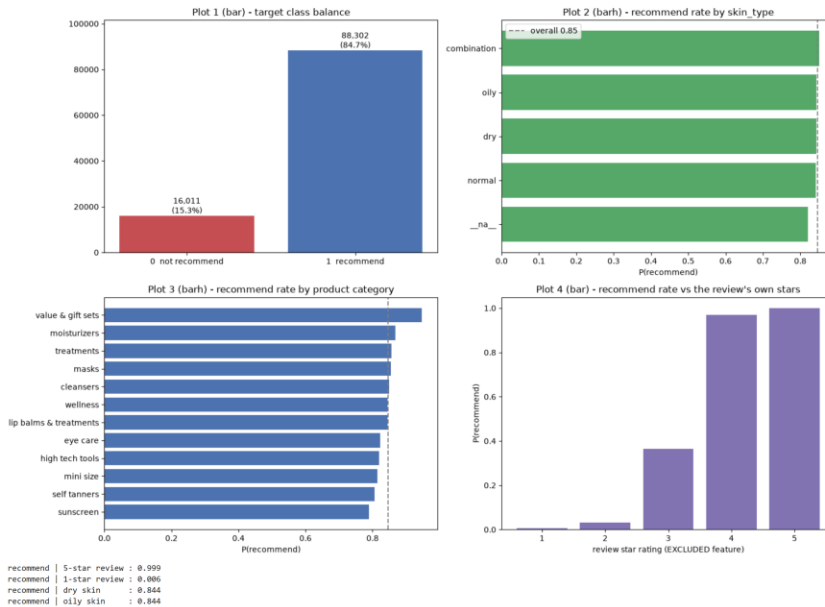
fig, ax = plt.subplots(2, 2, figsize=(15, 10))
vc = df.is_recommended.value_counts().sort_index()
b = ax[0,0].bar(vc.index, vc.values, color=["#c44e52", "#4c72b8"])
for f,v in zip(b, vc.values): ax[0,0].text(r.get_x()+r.get_width()/2, v, f'(v,)\n(v/len(df):.1%)", ha="center", va="bottom")
ax[0,0].set_title("Plot 1 (bar) - target class balance"); ax[0,0].set_ylim(0, vc.max()*1.15)

st = df.groupby("skin_type").is_recommended.agg(["mean","size"]).query("size >= 200").sort_values("mean")
ax[0,1].barh(range(len(st)), st["mean"].values, color="#55a868")
ax[0,1].set_yticks(range(len(st))); ax[0,1].set_yticklabels(st.index)
ax[0,1].axline(pos, ls="--", c="gray", label="overall (pos:.2%)"); ax[0,1].legend()
ax[0,1].set_xlabel("P(recommend)"); ax[0,1].set_title("Plot 2 (barh) - recommend rate by skin_type")

cr = df.groupby("secondary_category").is_recommended.agg(["mean","size"]).query("size >= 200").sort_values("mean")
ax[1,0].barh(range(len(cr)), cr["mean"].values, color="#4c72b8")
ax[1,0].set_yticks(range(len(cr))); ax[1,0].set_yticklabels(cr.index)
ax[1,0].axline(pos, ls="--", c="gray"); ax[1,0].set_xlabel("P(recommend)")
ax[1,0].set_title("Plot 3 (barh) - recommend rate by product category")

rr = df.groupby("rating").is_recommended.mean()
ax[1,1].bar(rr.index.astype(int).astype(str), rr.values, color="#8172b2")
ax[1,1].set_xlabel("review star rating (EXCLUDED feature)"); ax[1,1].set_ylabel("P(recommend)")
ax[1,1].set_title("Plot 4 (bar) - recommend rate vs the review's own stars")
plt.tight_layout(); plt.show()

print("recommended | 5-star review :", round(df.loc[df.rating==5,'is_recommended'].mean(), 3))
print("recommended | 1-star review :", round(df.loc[df.rating==1,'is_recommended'].mean(), 3))
print("recommended | dry skin :", round(df.loc[df.skin_type=='dry','is_recommended'].mean(), 3))
print("recommended | oily skin :", round(df.loc[df.skin_type=='oily','is_recommended'].mean(), 3))
```



```
In [10]: # Plot 5 - WHAT DO THE REVIEWS TALK ABOUT? (assignment Part V: comment characteristics / frequent terms)
from sklearn.feature_extraction.text import CountVectorizer
df["review_all"] = (df.review_title.fillna("") + " " + df.review_text.fillna("")).str.strip()
low = df.review_all.str.lower()

BUCKETS = {
    "texture / feel": "r[texture|greasy|sticky|light|w|absorb|silky|smooth|thick|runny|watery]",
    "skin outcome / effect": "r[skin|break|w|out|breakout|purge|ing|clear|w|up|hydrat|glow|smooth|w|skin|wrinkl|fine line|bright]",
    "irritation / reaction": "r[irritat|burn|sting|rash|allergic|sensitive|red|w|ness|itch|reaction]",
    "scent / fragrance": "r[scent|fragrance|scent|perfum|odour]",
    "price / value": "r[w|price|w|worth|expensive|pricey|affordab|value|b|cost|b|splurge|drugstore]",
    "packaging / amount": "r[packag|pump|bottle|jar|tube|dispenser|leak|amount|b|size|b]",
    "repurchase / recommend": "r[repurchas|buy again|holy grail|staple|recommend|will not buy|won't buy]",
}

n_c = len(df)
print(f"reviews: {n_c,} | median length (int(df.review_all.str.split().map(len).median())) tokens "
      f"p90 (int(df.review_all.str.split().map(len).quantile(.9)))w")
print(f"topic (non-exclusive): {26} % of reviews recommend-rate in bucket")
for name, pat in BUCKETS.items():
    n = low.str.contains(pat, regex=True, na=False)
    print(f"({name}:{26}) (n.mean():.1).3% (df.loc[n,'is_recommended'].mean():.2f)")

cv = CountVectorizer(ngram_range=(1,2), min_df=88, stop_words='english', strip_accents='unicode')
X = cv.fit_transform(low); vs = np.array(cv.get_feature_names_out()); fr = np.asarray(X.sum(0)).ravel()
print(f"top 25 terms overall:", ", ".join(vs[np.argsort(fr)[-1:-25]]))
neg = (df.is_recommended == 0).values
p_neg = np.asarray(X[neg].sum(0)).ravel(); p_pos = np.asarray(X[~neg].sum(0)).ravel(); i=1
lift = (p_neg/p_pos.sum()) / (p_pos/p_pos.sum())
print("most NOT-RECOMMEND-leaning: ", ", ".join(vs[np.argsort(lift)[-1:-15]]))
print("most RECOMMEND-leaning: ", ", ".join(vs[np.argsort(lift)[1:15]]))

reviews: 104,313 | median length 53 tokens (p90 115)

topic (non-exclusive) % of reviews recommend-rate in bucket
texture / feel 41.0% 0.89
skin outcome / effect 39.1% 0.90
irritation / reaction 22.7% 0.84
scent / fragrance 26.0% 0.84
price / value 12.0% 0.80
packaging / amount 18.0% 0.84
repurchase / recommend 21.2% 0.89
top 25 terms overall: skin, product, love, like, use, face, really, using, great, dry, feel, ve, cream, good, just, feels, moisturizer, used, definitely, recommend, doesn, amazin
& makeup, serum, don
most NOT-RECOMMEND-leaning: waste money, bring old, wanted like, did absolutely, disappointed product, wanted love, save money, returning, disappointing, meh, don waste, nope, ju
st okay, disappointing, won repurchasing
most RECOMMEND-leaning: new favorite, did disappoint, new staple, 10 10, amazing love, stuff amazing, lifesaver, excellent product, amazing using, definitely new, highly reco
mmend, far love, week love, yes yes, hooked
```

```
In [11]: # Plot 6 - HOW MUCH DOES EACH STRUCTURED ATTRIBUTE PREDICT THE TARGET? spread = mutual information
from sklearn.feature_selection import mutual_info_classif
def rec_spread(series):
    g = df.groupby(series).is_recommended.agg(["mean","size"]); g = g["size"] >= 200["mean"]
    return round(g.min(),2), round(g.max(),2), round(g.max()-g.min(),2)
rows = []
for name, s in [{"skin_type (4)", df.skin_type}, {"skin_tone (-13)", df.skin_tone},
                {"product category (12)", df.secondary_category}, {"brand (-79)", df.brand_name},
                {"price decile", pd.qcut(df.price_usd, 10, duplicates="drop")},
                {"loves_count decile", pd.qcut(df.loves_count, 10, duplicates="drop")},
                {"review-age decile", pd.qcut(df.review_age_days, 10, duplicates="drop")},
                {"review star rating (EXCLUDED)", df.rating}]:
    lo, hi, sp = rec_spread(s); rows.append((name, lo, hi, sp))
display(pd.DataFrame(rows, columns=["attribute", "recommend min", "recommend max", "spread"]))

MIX = pd.DataFrame({
    "log-price": np.log1p(df.price_usd.fillna(df.price_usd.median()))),
    "log-loves": np.log1p(df.loves_count.fillna(df.loves_count.median()))),
    "review_age_days": df.review_age_days.fillna(df.review_age_days.median())},
    {"skin_type_code": df.skin_type.astype("category").cat.codes,
     "skin_tone_code": df.skin_tone.astype("category").cat.codes,
     "category_code": df.secondary_category.astype("category").cat.codes,
     "brand_code": df.brand_name.astype("category").cat.codes,
     "rating_EXCLUDED": df.rating.fillna(df.rating.median())})
ni = mutual_info_classif(MIX, df.is_recommended, discrete_features=[3,4,5,6], random_state=RANDOM_SEED)
print("mutual information with is_recommended (higher = more informative):")
for k,v in sorted(zip(ni.columns, ni), key=lambda t: -t[1]): print(f"({k:16s}) {v:.4f}")
```

	attribute	recommend min	recommend max	spread
0	skin_type (4)	0.82	0.85	0.03
1	skin_tone (-13)	0.83	0.87	0.04

Figure 6.3

Figure 6.3 - Customer behaviour: topic-bucket recommend rates and mutual information with the target.

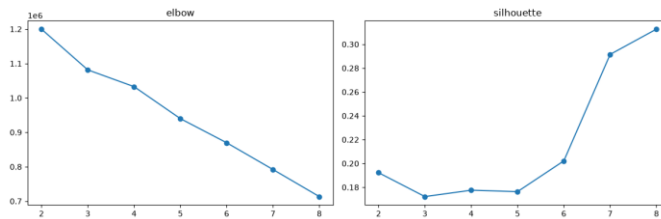


Figure 6.4

Figure 6.4 - Customer segmentation (K-Means): elbow and silhouette (notebook Appendix B).

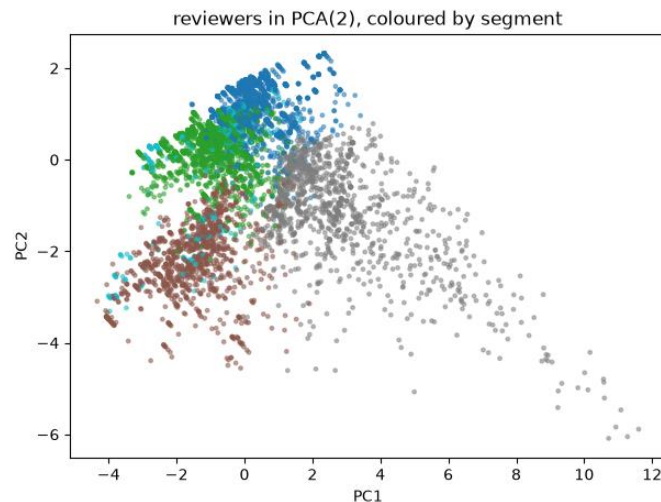


Figure 6.5

Figure 6.5 - Customer segmentation (K-Means): 5-segment profile and PCA scatter.

- **Class balance** 84.7 % recommend. Recommend-rate spread by skin type ~ 0.03 , by category ~ 0.16 , by brand ~ 0.40 , by price decile ~ 0.14 ; against the *excluded* star rating it runs $0.01 \rightarrow 1.00$. *Interpretation:* the structured customer-product fit is a real but weak signal; the star rating is essentially the label. *ML implication:* class_weight= "balanced", group the split on author_id, quarantine rating, expect the text to carry most of the skill.
- **Topic buckets (Figure 6.3)** - texture/feel ~ 42 % of reviews (recommend 0.89), skin outcome/breakouts ~ 39 % (0.90), irritation ~ 23 %, scent ~ 26 %, price ~ 13 %. **Mutual information** with the target: excluded rating 0.35, log_loves 0.037, log_price/brand 0.018, secondary_category 0.002, skin fields ~ 0 . *Interpretation:* what the reviewer writes about predicts the recommendation; demographics do not.
- **Customer segmentation (K-Means, Figures 6.4-6.5)** - reviewer $x_i = [R_i, F_i, M_i, \text{avg_price}_i, \text{recommend_rate}_i, \text{avg_rating}_i, C_{i1}..C_{ik}]$; 73 819 reviewers, $F \geq 2$ for 22 %, $k = 5$ from the elbow/silhouette. Descriptive only; not fed to the classifier.

6.6 Data leakage

1. **The review text is co-authored with the label** - is_recommended is a checkbox on the same form as the free-text box, so a text model partly *reads* the verdict; the ~0.965 is a retrospective upper bound (not pure leakage - the text also carries product experience, and tab+text still survives a temporal split at ~0.96).
2. **The review's own star rating IS the label in another column** - adding it -> ROC-AUC ~**0.985** (MI 0.35). Excluded. The product *average* rating is milder (0.768 -> 0.789) but also excluded as look-ahead.
3. **Engagement counts are post-publication** - kept for this retrospective model; a prospective variant that drops them falls to ROC-AUC ~0.66.

No preprocessing object is fitted on validation, test or user input - the deployed API loads the exact pipeline fitted on X_train.

6.7 Model development

Eight pipelines - the assignment's six for Application 3 (Logistic Regression, Decision Tree max_depth=12, Random Forest n_estimators=350, Linear SVM LinearSVC C=0.5, SGD(loss="log_loss") as the text-based linear classifier, HistGradientBoosting as the extra justified model) plus Complement Naive Bayes and the deployed Logistic Regression on tabular + text. All class-weighted; hyperparameters fixed; fitted on the training split only. **Split:** StratifiedGroupKFold(5) grouped on author_id (no reviewer spans splits) -> train 62 587 / val 20 862 / test 20 864, each at recommend rate 0.8465.

Representation ladder (one Logistic Regression, growing features, validation ROC-AUC): skin profile 0.536 -> + product 0.656 -> full tabular 0.768 -> text only 0.965 -> **tabular + text 0.966**.

Model	Representatio n	ROC-AUC	macro-F1	recall(0)	Accuracy
Logistic Regression (tab + text)	combo	0.966	0.861	0.885	0.919
HistGradient Boosting	tabular	0.811	0.664	0.678	0.771
Random Forest	tabular	0.810	0.686	0.611	0.806
Logistic Regression	tabular	0.768	0.622	0.679	0.722
Decision Tree	tabular	0.767	0.637	0.649	0.746
Linear SVM	tabular	0.767	0.622	0.676	0.723
SGD (log-loss, text)	text	0.965	0.856	0.885	0.916
Complement Naive Bayes	text	0.955	0.840	0.846	0.907

Model	Representation	ROC-AUC	macro-F1	recall(0)	Accuracy
baseline (majority class)		0.500	0.458	0.000	0.847

Tabular vs tabular + text - the assignment's key question. Both representations carry real skill (tabular ~0.77 linear / ~0.81 trees). The review text alone reaches ~0.965; adding it to the tabular block gives only a small, consistent lift (macro-F1 0.856 -> 0.861). They are **redundant more than complementary** - a customer who will withhold a recommendation has usually already said so in the review.

6.8 Evaluation and error analysis

18. Model comparison

```
In [21]: # --- 18a. representation ladder (one LogReg, growing feature set) ---
LADDER = {
    "A. skin profile only": ["skin_type", "skin_tone", "eye_color", "hair_color"],
    "B. + product (cat, brand, price, pop)": ["skin_type", "skin_tone", "eye_color", "hair_color",
                                             "secondary_category", "brand_name", "price_usd", "price_missing",
                                             "loves_count", "reviews"],
    "C. + review timing / listing (FULL TAB)": DEPLOY_TAB,
}

rows = []
for tag, cols in LADDER.items():
    pipe = Pipeline([("prep", make_tab(cols)),
                     ("clf", LogisticRegression(max_iter=1000, class_weight="balanced",
                                                random_state=RANDOM_SEED))]).fit(X_train, y_train)

    s = score(pipe, X_val, y_val)
    rows.append([tag, s["roc_auc"], s["macro_f1"], s["recall0"]])
# rung D (text) and E (tab+text)
for tag, kind in [{"D. text only (TF-IDF)", "text"}, ("E. tab + text (DEPLOYED)", "combo")]:
    pipe = Pipeline([("prep", prep_for(kind)),
                     ("clf", LogisticRegression(max_iter=1000, class_weight="balanced",
                                                random_state=RANDOM_SEED))]).fit(X_train, y_train)

    s = score(pipe, X_val, y_val)
    rows.append([tag, s["roc_auc"], s["macro_f1"], s["recall0"]])
ladder = pd.DataFrame(rows, columns=["representation", "roc_auc", "macro_f1", "recall(0)"]).round(3)
display(ladder)
print("product block adds :", round(ladder.roc_auc[1]-ladder.roc_auc[0], 3), "ROC-AUC over skin-only")
print("full tabular      :", ladder.roc_auc[2], ~ | text alone:", ladder.roc_auc[3],
      ~ | tab+text:", ladder.roc_auc[4])

representation roc_auc macro_f1 recall(0)
0 A. skin profile only 0.536 0.438 0.569
1 B. + product (cat, brand, price, pop) 0.656 0.522 0.634
2 C. + review timing / listing (FULL TAB) 0.768 0.622 0.679
3 D. text only (TF-IDF) 0.965 0.859 0.882
4 E. tab + text (DEPLOYED) 0.966 0.861 0.885

product block adds : 0.12 ROC-AUC over skin-only
full tabular      : 0.768 | text alone: 0.965 | tab+text: 0.966

In [22]: # --- 18b. the eight models ---
rows = []
for name, (pipe, kind) in fitted.items():
    s = score(pipe, X_val, y_val)
    rows.append([name, kind, s["accuracy"], s["macro_f1"], s["recall0"], s["roc_auc"]])
cmp = pd.DataFrame(rows, columns=["model", "representation", "accuracy", "macro_f1", "recall(0)", "roc_auc"]).round(3)
display(cmp.sort_values(["representation", "roc_auc"], ascending=[True, False]).reset_index(drop=True))

best_tab = cmp[cmp.representation=="tab"].sort_values("roc_auc").iloc[-1]
best_combo = cmp[cmp.representation=="combo"].sort_values("roc_auc").iloc[-1]
print(f"nbest tabular model : {best_tab['model']} (val ROC-AUC {best_tab['roc_auc']}, macro-F1 {best_tab['macro_f1']})")
print(f"nbest deployed model : {best_combo['model']} (val ROC-AUC {best_combo['roc_auc']}, macro-F1 {best_combo['macro_f1']})")
DEPLOY_NAME = "LogReg_tab_text"

model representation accuracy macro_f1 recall(0) roc_auc
0 LogReg_tab_text combo 0.919 0.861 0.885 0.966
1 HistGradientBoosting tab 0.771 0.664 0.678 0.811
2 RandomForest tab 0.806 0.686 0.611 0.810
3 LogisticRegression tab 0.722 0.622 0.679 0.768
4 DecisionTree tab 0.746 0.637 0.649 0.767
5 LinearSVC tab 0.723 0.622 0.676 0.767
6 SGD_log_text text 0.916 0.856 0.885 0.965
7 ComplementNB_text text 0.907 0.840 0.846 0.955

best tabular model : HistGradientBoosting (val ROC-AUC 0.811, macro-F1 0.664)
deployed model : LogReg_tab_text (val ROC-AUC 0.966, macro-F1 0.861)
```

Figure 6.6

Figure 6.6 - Customer behaviour: the representation ladder and the 8-model comparison table (notebook Section 18).

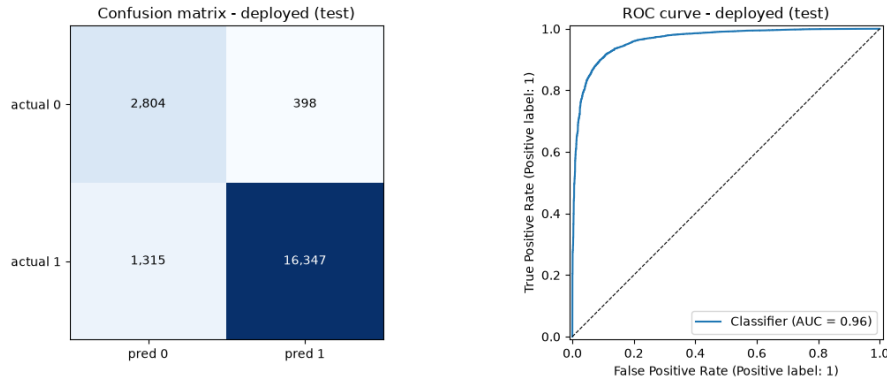


Figure 6.7

Figure 6.7 - Customer behaviour: classification report, confusion-matrix heatmap and ROC curve for the deployed Logistic Regression on tabular + TF-IDF text (notebook Section 19).

20. Error analysis

```
In [24]: err = df.iloc[test_idx].copy().reset_index(drop=True)
err["actual"] = y_test.values; err["pred"] = p_test; err["p_recommend"] = np.round(s_test, 3)
fn = err[(err.actual==0) & (err.pred==1)]
fp = err[(err.actual==1) & (err.pred==0)]
print(f"false negatives (missed 'not recommend'): {len(fn)}   false positives: {len(fp)}")

look = ["rating", "price_usd", "loves_count", "review_age_days", "text_len"]
display(pd.DataFrame({"all_test": err[look].mean(numeric_only=True),
                      "false_negatives": fn[look].mean(numeric_only=True),
                      "false_positives": fp[look].mean(numeric_only=True)}).round(2))
print(f"nFN with a 4-5 star rating: {:.0%} (rating is not a feature - these say 'recommend' in stars".format(
    (fn.rating >= 4).mean()), "but ticked 'no'")
print("sample missed 'not recommend' reviews:")
for t in fn.sort_values("p_recommend", ascending=False).review_text.head(3):
    print("  ", (t[:160] + "...") if isinstance(t, str) else t)
```

false negatives (missed 'not recommend'): 398 false positives: 1315

	all_test	false_negatives	false_positives
rating	4.30	2.84	4.06
price_usd	49.58	51.13	45.81
loves_count	29801.48	27108.84	34551.87
review_age_days	721.37	731.22	765.56
text_len	336.80	348.50	390.96

FN with a 4-5 star rating: 22% (rating is not a feature - these say 'recommend' in stars but ticked 'no')

sample missed 'not recommend' reviews:

- I love how this makes my skin feel! My skin looks incredible. I have fine lines larger pores and some unevenness and while this didn't reverse it it sure is he...
- Complimentary from Murad Skincare. By far my favorite was to clear acne! I get quite a few breakouts and haven't then able to find something that works for me f...
- Sun protection 101 such a great high quality high spf sunscreen! So happy to have been able to try it out it's so great in coverage specially for the California...

Figure 6.8

Figure 6.8 - Customer behaviour: false-negative / false-positive feature comparison with sample missed reviews (notebook Section 20).

Deployed Logistic Regression (tab + text), held-out test (N = 20 864): accuracy **0.918**, ROC-AUC **0.964**, recall on “does not recommend” **0.876** (precision 0.681, F1 0.766). Single-representation on the same test set: tabular-only 0.801, text-only 0.963. **Confusion matrix** **[[2 804, 398], [1 315, 16 347]]**: the 1 315 false negatives are customers who will not recommend but are predicted as recommending - a misleading product page; the 398 false positives are happy customers flagged for QA - cheap. **Most important metric: recall on class 0**. Accuracy is not the headline - the majority baseline already scores 0.847 while catching zero unhappy reviewers. **Error analysis**: 22 % of the false negatives carry a 4-5 star rating - a positive-reading review with a hidden veto, which is the irreducible ceiling for a text model.

6.9 Business interpretation

Whether a customer recommends is **mostly in what they write** (text ~ 0.965); the structured customer-product fit is a real but weaker, redundant signal (~ 0.8). Uses: **review-consistency QA** (flag “5 stars” + “would not repurchase”), **cold-start ranking** from the tabular-only model (no text needed, ~ 0.8), **merchandising** on low-recommend categories. The model is retrospective (it needs the review), so its value is the *structured* view, not the small accuracy edge over a plain text classifier.

6.10 Persistence and deployment

Persisted as customer_behaviour/model/model_pipeline.joblib (ColumnTransformer + TfidfVectorizer + LogisticRegression, ~ 2 MB) + feature_names.joblib + feature_means.joblib (linear-SHAP reference) + input_schema.json. POST /predict returns {prediction, confidence, p_recommend, threshold, review_terms, signals, contributions, model, representation}, where contributions is an exact linear-SHAP decomposition. Section 23 inference test: reload == in-memory; positive review $\rightarrow$ p_recommend = 0.9376, negative $\rightarrow$ 0.0026. Deployment: Section 8, Appendix D.3, Appendix E.

7. Comparison of the Three Intelligent Systems

Aspect	Diabetes	House Price	Customer Behaviour
Problem type	Classification	Regression	Classification
One observation	one survey respondent	one property listing	one product review
Target	Diabetes_binary in {0,1}	Price in R^+ (as $\log_{10}$)	is_recommended in {0,1}
Input representation	$R^{\{N \times 23\}}$ scaled + passthrough	$R^{\{N \times 92\}}$ impute + scale + one-hot (sparse)	$R^{\{N \times 28\,733\}}$ tabular + TF-IDF text (sparse)
Data-quality issues	23 899 duplicate rows; 17 % imbalance; BMI tail	70-84 % missing size fields; $\sim 11\,000$ impossible prices; near-zero numeric correlation; title leaks price	~ 10 % blank target; 85 % imbalance; missing skin-profile fields; text co-authored with the label
Best model	Random Forest	Random Forest	Logistic Regression (tab + text)
Main metric	recall on diabetic class - 0.735 at ROC-AUC 0.809	RMSE 26 873 / R^2 0.186	recall on “does not recommend” - 0.876 at ROC-AUC 0.964
Web deployment	Yes	Yes	Yes (3-step wizard)
Mobile deployment	Yes	Yes	Yes
Main limitation	recall bought with 0.36 precision; survey self-report	weak price signal ($R^2 \sim 0.19$); needs finer location than province	text co-authored with the tick (~ 0.96 partly leakage; leak-safe tabular ~ 0.8);

Aspect	Diabetes	House Price	Customer Behaviour
			retrospective

Discussion. The three datasets need three different representations: a plain scaled matrix, an imputed one-hot matrix, and a matrix concatenated with sparse text vectors. The common preprocessing is median imputation fitted on train, scaling, a stratified or author-grouped split, and one persisted Pipeline; the application-specific parts are the log1p target and *_missing indicators (house price), the duplicate drop and engineered risk flags (diabetes), and the OneHotEncoder(min_frequency) + TfidfVectorizer + author-grouped split + leakage section (customer behaviour). The targets differ because the questions differ (“is this true?” / “how much?” / “will they endorse?”), which is why a classifier is judged on the confusion matrix and ROC-AUC while a regressor is judged on error magnitude and variance explained. Diabetes is the easiest to deploy (23 numeric inputs, no text); customer behaviour is the most compute-heavy (a ~28 700-dim sparse tabular+text matrix, a TF-IDF vocabulary to ship, and eight models).

8. Deployment Architecture

```
# FastAPI POST /predict Endpoint Implementation
@app.post('/predict', response_model=PredictResponse)
def predict(request: PredictRequest):
    df_raw = pd.DataFrame([request.dict()])
    # Apply identical loaded preprocessing pipeline
    proba = pipeline.predict_proba(df_raw)[0, 1]
    return PredictResponse(
        prediction='diabetic' if proba >= threshold else 'not diabetic',
        confidence=round(float(proba), 4)
    )
```

All three applications follow the same inference path:

User input -> API request -> validation -> SAME preprocessing (loaded)
-> saved model -> prediction -> JSON

1. **Trained model** - loaded from model/model_pipeline.joblib, **never retrained** at request time.
2. **Preprocessing pipeline** - the exact ColumnTransformer / Pipeline fitted on the training split, saved inside the same object.
3. **API endpoint** - FastAPI POST /predict (plus GET /healthz, GET /model-info, GET /questions for the clients to render their forms).
4. **Input validation** - a Pydantic schema (types, ranges); missing optional fields are median-imputed by the pipeline.
5. **Prediction** - pipeline.predict_proba (classification) or expm1(pipeline.predict) (house-price regression).
6. **Result** - JSON: {prediction, confidence} / {predicted_price} / {prediction, p_recommend, contributions, ...}.
7. **Web UI** - React + Vite form -> fetch -> result screen with a plain-language verdict.
8. **Mobile UI** - Flutter form -> REST call -> result screen. Training != Inference: the mobile app trains nothing, it calls the same endpoint.

Data-leakage rule. The service loads the preprocessing pipeline *fitted on training data* and **never calls .fit()** on a request; a scaler re-fitted on one request would standardise it against itself. Training and inference apply identical transforms - verified by each notebook's Section 23 inference test (disk == in-memory).

Per-app request/response summary:

App	Endpoint	Example input	Example output
Diabetes	POST :8000/predict	{ Age, Sex, HighBP, HighChol, BMI, GenHlth, DiffWalk, ... }	{ "prediction": "at risk", "confidence": 0.82, "band": "High" }
House price	POST :8002/predict	{ Area, "Property Type", Province, district, Bedrooms, ... }	{ "predicted_price": 2540.82, "price_per_m2": 32.28, "currency": "million VND" }
Customer behaviour	POST :8000/predict	{ skin_type, secondary_category, brand_name, price_usd, review_title, review_text } (no rating)	{ "prediction": "not recommend", "p_recommend": 0.0026, "review_terms": { ... }, "contributions": [...] }

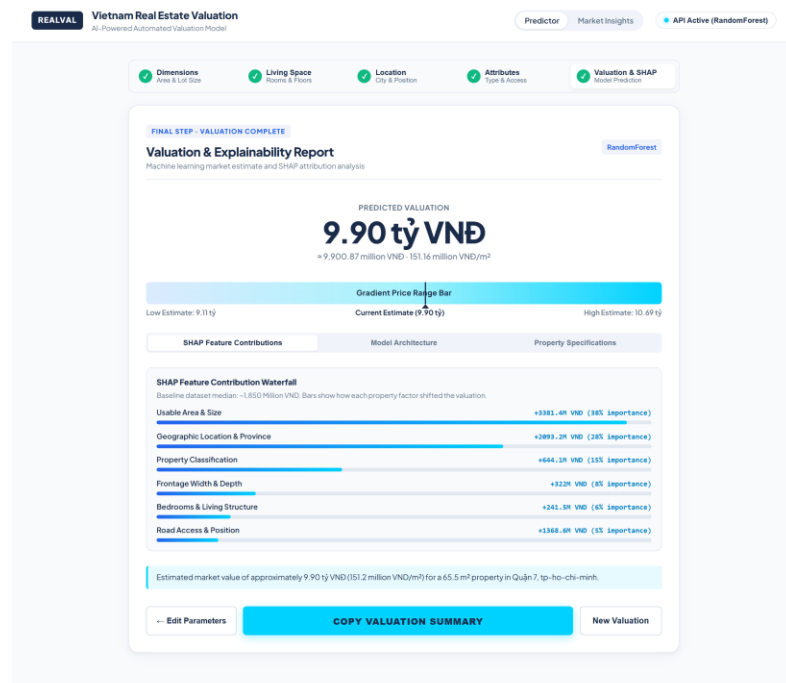


Figure 8.1

Figure 8.1 - Representative end-to-end screen (house-price web client). The user enters property attributes; the page POSTs them to /predict; the API runs the loaded pipeline and Random Forest,

inverts log1p with expm1, and returns the price; the result screen shows the value (about 9.90 billion VND), a low-high range, a SHAP contribution waterfall and a plain-language summary.

The complete web and mobile screenshots (input screen, result screen and, for the web, the FastAPI /docs route) for every application, each with a Figure-explanation note as the assignment requires, are in **Appendix D** (web) and **Appendix E** (mobile). Every mobile run logged a POST /predict 200 on the server - the required communication evidence.

9. Reproducibility

Item	Diabetes	House Price	Customer Behaviour
Python / OS	3.13 / Windows 11	3.13 / Windows 11	3.13 / Windows 11
Key libraries	scikit-learn 1.9.0, numpy, pandas, scipy, matplotlib; fastapi + uvicorn + pydantic 2	+ xgboost	+ React 18 + Vite 5; Flutter 3.19+
Random seed	RANDOM_SEED = 42 (numpy + random, every split and estimator)	same	same
Dataset	Kaggle alexteboul/..., diabetes_012_health_indicators_BRFSS2015.csv - committed	Kaggle qmanhbeo/... (CC BY-NC 4.0) - not committed (~210 MB)	Kaggle nadyinky/... (CC0) - not committed (~505 MB); reviews_500-750.csv + product_info.csv
Rows (raw -> clean)	253 680 -> 229 781	236 226 -> 201 654	116 262 -> 104 313
Split	stratified 70/15/15 (160 846 / 34 467 / 34 468)	70/15/15 (141 157 / 30 248 / 30 249)	StratifiedGroupKFold(5) on author_id (62 587 / 20 862 / 20 864)
Preprocessing	SimpleImputer(median) -> StandardScaler (8 cols) + passthrough (15)	SimpleImputer(median) -> log1p -> StandardScaler; OneHotEncoder(min_frequency=50)	+ OneHotEncoder(min_frequency=25); TfidfVectorizer(ngram_range=(1,2), min_df=10, max_features=40000, sublinear_tf, stop_words="english") - all fitted on train only
Feature dim d	23	92	28 733 (sparse)
Model	RandomForestClassifier (max_depth=12, min_samples_leaf=20, class_weight="balanced", random_state=42)	RandomForestRegressor(random_state=42) on log(1 + Price)	LogisticRegression(C=1.0, class_weight="balanced", random_state=42) on

Item	Diabetes	House Price	Customer Behaviour
			tabular + TF-IDF
Test metrics	Acc 0.725 / recall(1) 0.735 / ROC-AUC 0.809 / conf [[20612,7897],[1579,4380]]	MAE 10 626 / RMSE 26 873 / R ² 0.186	Acc 0.918 / macro-F1 0.858 / ROC-AUC 0.964 / recall(0) 0.876 / conf [[2804,398],[1315,16347]]
Saved artifacts	model_pipeline.joblib, feature_names.joblib, input_schema.json	same	+ feature_means.joblib
Reproduce	pip install -r requirements.txt -> run the notebook (writes model/) -> python -m uvicorn api.main:app -> npm --prefix web run dev -> flutter run in mobile/		

```
import numpy as np, random
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED); random.seed(RANDOM_SEED)
```

Repository layout: Appendix A. Notebook structure: Appendix B. API specification: Appendix C. Submission checklist: Appendix F.

10. Final Comparative Discussion

The comparison table and the “how do the datasets / representations / metrics differ” answers are in Section 7. This section covers the parts that go beyond that table.

Which model performed best, and did it deploy best? Random Forest (diabetes, house price) and Logistic Regression on tabular + text (customer behaviour). In all three the best model was also the best to deploy: each is a single scikit-learn Pipeline, CPU-only, milliseconds per prediction, one small artifact.

Forms of data leakage considered. (a) A scaler, encoder or vectoriser re-fitted at inference - avoided by loading the fitted pipeline and never calling .fit(). (b) A duplicate row shared by train and test - the 23 899 diabetes duplicates are dropped *before* the split. (c) A column that embeds the target - the house-price Title contains the price string and is dropped. (d) The review star rating, which is the recommend label - quarantined; adding it back pushes ROC-AUC to ~0.985. (e) Engagement counts that only exist after publication - kept for the retrospective customer-behaviour model but flagged, and a prospective variant that drops them falls to ~0.66.

What information is lost and preserved during representation. One-hot encoding loses the ordering of nominal categories but preserves identity; log1p on money columns loses the raw scale but preserves rank and makes the target near-symmetric; TF-IDF loses word order and

syntax but preserves which terms are distinctive; min_frequency merges rare categories, losing tail identity to control dimensionality. What is preserved in every case is the information the chosen model can actually use.

Remaining limitations. Diabetes: recall is bought with 0.36 precision, and the data is one country and one year of self-reported survey answers. House price: the model explains only ~19 % of variance because the dataset has almost no numeric signal and coarse location; a better result needs street-level location and price-per-m² references. Customer behaviour: the headline text score is partly leakage and the model is retrospective (it needs the finished review).

What we would improve with more time. Calibrated thresholds and cost-weighted decisions for diabetes; joining external location / price-per-m² data for house price; an aspect-level text model and a multi-category pull (to restore RFM Frequency variance) for customer behaviour; and a shared internal library for the three build_features() / schema / inference code paths so a change is made once.

11. Conclusion

Raw Data -> Clean -> Represent -> Learn -> Evaluate -> Persist -> Deploy

A deployable intelligent system is **data + representation + learning + evaluation + software + deployment + user interaction**, not just a trained model.

1. **Main lesson** - most of the work and most of the risk is in turning raw data into a correct, reproducible representation; the model is a small final step.
2. **Biggest technical challenge** - building **one** preprocessing object that is fitted once on training data and applied identically in the notebook, the API and the tests.
3. **Most important data-representation issue** - deciding what each raw column *is* (a real feature, a leak, a post-hoc signal, an id) before encoding it - most visible in Application 3's leakage analysis.
4. **Most important ML lesson** - accuracy is the wrong headline on imbalanced or weak-signal data; the confusion matrix, recall on the class that matters, and R² versus a naive baseline tell the real story.
5. **Most important deployment lesson** - Training != Inference: the service loads the fitted pipeline and never re-fits.
6. **One future improvement** - a shared internal library for the three feature-build / schema / inference code paths.

Appendix A - Repository Structure

Directory / File Path	Description & Purpose
assignment_02/	Root repository directory containing all application modules and deliverables.
├─ report/	Report documentation and screenshot evidence repository.
├─ Report_Deliverable.docx	Main deliverable report in Microsoft Word format.
├─ Report_Deliverable.pdf	Final compiled deliverable report in PDF format.
└─ screenshots/	Contains all Web, Mobile, and Notebook screenshots, indexed in README.md.

└─ diabetes/	Application 1 — Diabetes Screening System (Binary classification).
└─ house_price/	Application 2 — House Price Prediction System (Continuous price regression).
└─ customer_behaviour/	Application 3 — E-Commerce Customer Interest Discovery (Tabular + Text).
└─ data/	Kaggle raw CSV datasets and data/README.md download instructions.
└─ notebook/	23-section Jupyter notebook executing top-to-bottom without errors (*.ipynb).
└─ model/	Persisted ML artifacts (model_pipeline.joblib, feature_names.joblib, input_schema.json).
└─ api/	FastAPI REST service (main.py, schema.py, inference.py, config.py, requirements.txt).
└─ web/	React 18 + Vite frontend single-page web client (src/, package.json).
└─ mobile/	Flutter mobile client application (lib/, pubspec.yaml).
└─ README.md	Master reproduction guide explaining how to execute notebooks and start services.

Appendix B — Jupyter Notebook Architecture & Section Specification

All three Jupyter notebooks (diabetes/notebook/diabetes.ipynb, house_price/notebook/house_price.ipynb, and customer_behavior/notebook/customer_behavior.ipynb) follow an identical 23-section architecture. Every section contains executed Python code cells with zero execution errors, followed by written markdown interpretations. The complete section specification and verification contract is detailed below.

Section # & Name	Purpose & Workflow Phase	Deliverable & Verification Output
0. Header & Setup	Environment fixation & reproducibility setup	Library versions printed; RANDOM_SEED = 42 set for numpy, random, and sklearn.
1. Problem Definition	Formal supervised task specification	Supervised task type (Classification / Regression), input vector x_i , target y_i , domain goal.
2. Dataset Source	Provenance & metadata documentation	Kaggle URL, dataset license, collection methodology, and local storage path.
3. Dataset Loading	Raw data ingestion	CSV loading into pandas.DataFrame, delimiter check, df.head() display.
4. Dataset Inspection	Initial structural audit	df.shape, column dtypes, summary statistics (df.describe()), missing/duplicate counts.
5. Data-Quality Analysis	Systematic data issue identification	Compact issue table: missing values, duplicates, invalid entries, class imbalance.
6. Missing-Value Analysis	Feature-level missingness assessment	Column-level missing percentages and strategy selection (drop vs. median imputation).
7. Duplicate Analysis	Record uniqueness verification	Exact row duplicates and domain key duplicates checked; deduplication executed.
8. Invalid-Value Analysis	Domain boundary verification	Physiologically/physically impossible values identified (e.g. Glucose=0, Price<=0) -> NaN.
9. Outlier Analysis	Extreme value detection & treatment	IQR boxplot analysis; capping at 99th percentile vs. robust tree handling

		justified.
10. Exploratory Data Analysis	Feature & target pattern discovery	>= 3 meaningful plots (Target dist, Correlation heatmap, Scatter/Boxplots) + O/I/ML notes.
11. Feature Types	Feature classification & selection	Categorization: continuous, categorical, text, ID, target; leakage features dropped.
12. Data Representation	Core Lecture 02 link demonstration	Chain: CSV -> DataFrame -> Clean Matrix -> Scaled/Encoded -> X (Nxd). Text embeddings E.
13. Feature Engineering	Domain-specific feature creation	Engineered indicators (CardioRisk, TotalUnhealthyDays, log1p(Price), TF-IDF features).
14. Train/Val/Test Split	Independent dataset partitioning	Stratified/Grouped split (70/15/15), independent test set held out, data leakage prevented.
15. Preprocessing Pipeline	Unified ColumnTransformer creation	Single pipeline fitted on training data only (fit(X_train)), combining impute, scale, encode.
16. Baseline Model	Performance floor benchmark	Naive baseline (DummyClassifier / DummyRegressor) establishing minimum score to beat.
17. Model Training	Multi-model training phase	5-8 candidate models trained on identical preprocessed training data (X_train).
18. Model Comparison	Validation set evaluation	Summary metric comparison table across all models evaluated on validation split (X_val).
19. Evaluation	Final model assessment	Comprehensive evaluation of winner on held-out test set (Confusion Matrix, ROC-AUC, RMSE, R ²).
20. Error Analysis	Model failure diagnosis	Inspection of worst predictions / misclassifications; identification of remaining representation gaps.
21. Model Selection	Multi-criteria final selection	Trade-off analysis: Predictive performance vs. interpretability vs. latency vs. memory size.
22. Model Persistence	Artifact serialization	joblib.dump(pipeline, model_pipeline.joblib) bundling preprocessor + model into 1 file.
23. Inference Test	Disk reloading & API contract test	Reloading pipeline from disk, passing raw unprocessed JSON row, confirming identical prediction.

Appendix C - API Endpoint Specification

Workflow for every application: JSON input -> Pydantic validation -> loaded preprocessing -> model -> JSON output. The service never calls .fit().

	Diabetes	House price	Customer behaviour
Framework	FastAPI + Uvicorn	FastAPI + Uvicorn	FastAPI + Uvicorn
Port	8000	8002	8000
Prediction endpoint	POST /predict (optional ?include=explain,similar,whatif)	POST /predict	POST /predict, POST /predict/batch

	Diabetes	House price	Customer behaviour
Support endpoints	GET /healthz, /model-info, /questions, /metrics, /threshold-curve	GET /healthz	GET /healthz, /model-info, /questions, /samples
Prediction function	pipeline.predict_proba	expm1(pipeline.predict)	pipeline.predict_proba + linear-SHAP
Output keys	prediction, confidence, band, contributions	predicted_price, price_per_m2, currency, model, contributions	prediction, confidence, p_recommend, threshold, review_terms, signals, contributions, model, representation

The interactive Swagger UI is served at each service's /docs route, with POST /predict expanded, and documents the request and response schema.

Appendix D - Web Application Report

The three web clients share one design - React 18 + Vite, axios, **no model in the browser**: each renders its form from GET /questions, POSTs the entered fields to /predict, and shows the returned prediction with a plain-language interpretation.

D.1 Web Application - Diabetes

Item	Value
Framework	React 18 + Vite; the dev server proxies /api to the FastAPI service
API endpoint	POST http://localhost:8000/predict
Input variables	21 BRFSS items (Age, Sex, HighBP, HighChol, CholCheck, BMI, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, GenHlth, MentHlth, PhysHlth, DiffWalk, Education, Income) plus optional height_cm / weight_kg
Validation rules	Pydantic schema on the API; any omitted field is median-imputed by the loaded pipeline and the result is marked less certain
Preprocessing used	the saved Pipeline fitted on training data (engineer() then SimpleImputer(median) -> StandardScaler on 8 columns, 15 passthrough); never re-fitted
Loaded model	RandomForestClassifier(class_weight="balanced") inside the same pipeline object
Example request	{ "Age": 9, "Sex": 1, "HighBP": 1, "HighChol": 1,

Item	Value
	"BMI": 34, "GenHlth": 4, "DiffWalk": 1, "Smoker": 1 }
Example response	{ "prediction": "at risk", "confidence": 0.82, "band": "High", "contributions": [...] }

Diabetes Screening API

1.0.0 OAS 3.1

/openapi.json

Assignment 02 · Application 1 — questionnaire → risk band, with explanation, similar cases, what-if analysis and an operator dashboard.

default

POST /predict Screen one respondent

Try it out

Parameters

Name	Description
include string (query)	comma list: explain,similar,whatif,counterfactual Default value : include
log boolean (query)	set false for what-if probes that should not be recorded Default value : true true

Request body required

Example Value | Schema

```
{
  "Age": 0,
  "AnyHealthcare": 1,
  "CholCheck": 1,
  "DiffWalk": 1,
  "Education": 0,
  "Fruits": 0,
  "GenHlth": 0,
  "HeartDiseaseorAttack": 0,
  "HighBP": 1,
  "HighChol": 1,
  "HeavyAlcoholConsump": 0,
  "Income": 1,
  "MentHlth": 10,
  "NoDocCost": 0,
  "PhysActivity": 0,
  "PhysHlth": 10,
  "Sex": 1,
  "Smoker": 1,
  "Stroke": 0,
  "Veggies": 1,
  "height_cm": 175,
  "session_id": "demo-1",
  "weight_kg": 104
}
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header: Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

POST	/explain	Explain Route	⌵
POST	/similar	Similar Route	⌵
POST	/whatif	Whatif Route	⌵
POST	/counterfactual	Counterfactual Route	⌵
GET	/history	History Route	⌵
GET	/metrics	Metrics Route	⌵
GET	/threshold-curve	Threshold Curve Route	⌵
GET	/threshold-recommend	Threshold Recommend Route	⌵

Figure D3 - Diabetes FastAPI Swagger UI (/docs) endpoint specification

Diabetes screening
Screening
Operator

How likely is it that you have diabetes?

Answer a short health questionnaire and get an estimate from a model trained on a large public health survey, plus what drove the number and what could change it. It does not replace a test or a doctor.

Health questionnaire

Answer what applies to you. Anything you leave blank is filled with a typical value and the result is marked as less certain. Fields marked * are required.

ABOUT YOU

Age group *
Select...
Sex *
Select...

Highest level of school completed
Prefer not to say
Annual household income
Prefer not to say

BODY

Height (cm)
120-230
Weight (kg)
30-320

Used to compute BMI. Leave blank if unknown. Used to compute BMI.

BMI (if already known)

leave blank to compute from height & weight
Optional — leave blank to compute from height and weight.

CONDITIONS

Have you been told you have high blood pressure?

Yes
No

Have you been told your blood cholesterol is high?

Yes
No

Have you ever been told you had a stroke?

Yes
No

Have you ever had coronary heart disease or a heart attack?

Yes
No

Do you have serious difficulty walking or climbing stairs?

Yes
No

CHECKS & CARE

Have you had a cholesterol check in the past 5 years?

Yes
No

Do you have any kind of health-care coverage?

Yes
No

In the past year, did you need to see a doctor but could not because of cost?

Yes
No

LIFESTYLE

Have you smoked at least 100 cigarettes in your life?

Yes
No

Are you a heavy drinker (≥ 14 drinks/week for men, ≥ 7 for women)?

Yes
No

Any physical activity in the past 30 days (outside your job)?

Yes
No

Do you eat fruit at least once per day?

Yes
No

Do you eat vegetables at least once per day?

Yes
No

HOW YOU FEEL

In general, would you say your health is:
Prefer not to say
Days in the last 30 your mental health was not good
0

Days in the last 30 your physical health was not good
0

See my estimate

Your earlier screenings

Figure D1

Figure D1 - Diabetes web: the input questionnaire with a valid example entered.

How likely is it that you have diabetes?

Answer a short health questionnaire and get an estimate from a model trained on a large public health survey, plus what drove the number and what could change it. It does not replace a test or a doctor.

Health questionnaire

Answer what applies to you. Anything you leave blank is filled with a typical value and the result is marked as less certain. Fields marked * are required.

ABOUT YOU

Age group *

65-69

Sex *

Male

Highest level of school completed

Prefer not to say

Annual household income

Prefer not to say

BODY

Height (cm)

120-230

Weight (kg)

30-320

Used to compute BMI. Leave blank if unknown.

Used to compute BMI.

BMI (if already known)

34

Optional — leave blank to compute from height and weight.

CONDITIONS

Have you been told you have high blood pressure?

Yes

No

Have you been told your blood cholesterol is high?

Yes

No

Have you ever been told you had a stroke?

Yes

No

Have you ever had coronary heart disease or a heart attack?

Yes

No

Do you have serious difficulty walking or climbing stairs?

Yes

No

CHECKS & CARE

Have you had a cholesterol check in the past 5 years?

Yes

No

Do you have any kind of health-care coverage?

Yes

No

In the past year, did you need to see a doctor but could not because of cost?

Yes

No

LIFESTYLE

Have you smoked at least 100 cigarettes in your life?

Yes

No

Are you a heavy drinker (≥14 drinks/week for men, ≥7 for women)?

Yes

No

Any physical activity in the past 30 days (outside your job)?

Yes

No

Do you eat fruit at least once per day?

Yes

No

Do you eat vegetables at least once per day?

Yes

No

HOW YOU FEEL

In general, would you say your health is:

Fair

Days in the last 30 your mental health was not good

0

Days in the last 30 your physical health was not good

0

See my estimate

82%

High risk

High risk — refer for a confirmatory blood test.

This is a screening aid, not a diagnosis. Only a clinician can confirm diabetes.

You answered 19 of 21 questions. Blanks were filled with typical values.



People like you in the survey

The 5 survey respondents whose full answer pattern is closest to yours, found with the same random forest that made the estimate.

4 of 5

had diabetes or pre-diabetes

1

M, age 65-69, BMI 35, GenHth 4 (high BP, high chol, smoker, inactive, walk difficulty)

diabetes / pre-diabetes - similarity 31%

2

M, age 65-69, BMI 32, GenHth 4 (high BP, high chol, smoker, inactive, walk difficulty)

diabetes / pre-diabetes - similarity 23%

3

M, age 55-59, BMI 32, GenHth 4 (high BP, high chol, smoker, walk difficulty)

diabetes / pre-diabetes - similarity 20%

4

M, age 65-69, BMI 34, GenHth 5 (high BP, high chol, smoker, inactive, walk difficulty)

diabetes / pre-diabetes - similarity 20%

5

M, age 75-79, BMI 35, GenHth 4 (high BP, high chol, smoker, inactive, walk difficulty)

no diabetes - similarity 19%

Profiles are de-identified survey rows. A high count here is not a diagnosis - it reflects who else answered similarly.

What could change the estimate

Re-scoring your answers with one habit changed at a time. These are patterns in the survey data, not promises about your health.

If you had never smoked regularly

82% — 82%

-1 pts

Figure D2

Figure D2 - Diabetes web: the result screen - 82 % High risk, SHAP force plot, the five most similar survey respondents, and a what-if BMI slider.

Figure explanation (D1-D2). The user answers the questionnaire (D1); blank fields are median-imputed. The service returns $p(\text{diabetes}) = 0.82$ and the page (D2) shows the band “High risk - refer for a confirmatory blood test”, a SHAP force plot of which answers moved the estimate, the five most similar survey respondents, and a what-if BMI slider - a high-but-not-certain probability is deliberately surfaced as “refer for a test”, not a diagnosis. The /docs route documents the same request and response schema.

D.2 Web Application - House price

Item	Value
Framework	React 18 + TypeScript + Vite; axios
API endpoint	POST http://localhost:8002/predict
Input variables	Area (required, m^2); optional Width, Length, Bedrooms, Bathrooms, Floors, "Alley Width", "Agent Listing Count", "Property Type", Position, Direction, "Road Type", Province, "Agent Role", district
Validation rules	Pydantic schema - Area > 0 required; missing optional numerics median-imputed; unknown categoricals map to an all-zero one-hot row
Preprocessing used	saved ColumnTransformer - SimpleImputer(median) -> log1p -> StandardScaler (numeric), OneHotEncoder(handle_unknown="ignore", min_frequency=50) (categorical); never re-fitted
Loaded model	RandomForestRegressor trained on $\log(1 + \text{Price})$; the response inverts with $\exp m1$
Example request	{ "Area": 78.7, "Width": 4.0, "Bedrooms": 3, "Bathrooms": 2, "Floors": 2, "Property Type": "house", "Province": "an-giang", "district": "Rach Gia" }
Example response	{ "predicted_price": 2540.82, "price_per_m2": 32.28, "currency": "million VND", "model": "RandomForest", "contributions": [...] }

House-Price Prediction API

1.0.0

OAS 3.1

/openapi.json

Assignment 02 - Application 2: Automated Valuation Model for Vietnam Real Estate. Loads a persisted sklearn Pipeline (ColumnTransformer + RandomForestRegressor). Training != Inference: no fitting at runtime.

Health

GET

/healthz

Healthz

Inference

POST

/predict

Predict

Perform real estate valuation inference based on property attributes.

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

Schema

```
{
  "Agent Listing Count": 1,
  "Agent Role": "Chính chủ",
  "Alley Width": 3.5,
  "Area": 75.7,
  "Bathrooms": 2,
  "Bedrooms": 3,
  "Direction": "Nam",
  "Floors": 2,
  "Length": 19.6,
  "Position": "Đường chính",
  "Property Type": "Nhà riêng",
  "Province": "An-giang",
  "Road Type": "Đường nhựa",
  "Width": 4,
  "District": "Mạch Giã",
  "Ward": "Phường An Hòa"
}
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "predicted_price": 0, "price_per_m2": 0, "formatted_price_billion": "string", "currency": "million VND", "model_name": "string", "interpretation": "string", "feature_contributions": [{ "name": "string", "label": "string", "impact_million": 0, "direction": "string", "importance_pct": 0 }], "model_metadata": { "algorithm": "string", "n_estimators": 100, "max_depth": 16, "features_count": 18, "transformed_features": 117, "target_transform": "logip(Price) -> expm(y)", "zero_leakage": true } }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

Schemas

FeatureContribution

Expand all

object

HTTPValidationError

Expand all

object

HealthOut

Expand all

object

Figure D6 - House-price FastAPI Swagger UI (/docs) endpoint specification

© 2024 REALVAL - Vietnam Real Estate Valuation Engine. Powered by Random Forest Regressor.

Figure D3

Figure D3 - House-price web: the valuation wizard step 1 with the “District 7 Apartment” preset filled.

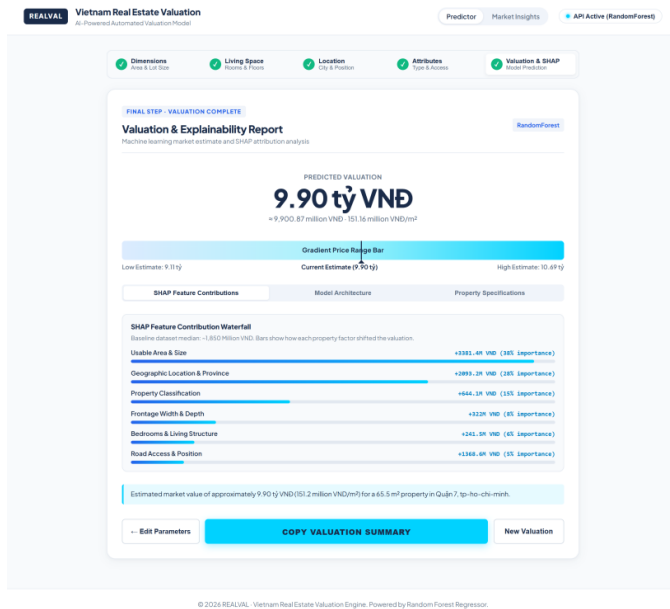


Figure D4

Figure D4 - House-price web: the result screen - predicted about 9.90 billion VND, a low-high range, a SHAP contribution waterfall and a plain-language summary.

Figure explanation (D3–D4). The user enters property attributes (D3; only Area > 0 is required). The service returns predicted_price and the page (D4) shows about 9.90 billion VND (about 151 million VND/m²) with a low-high range and a SHAP waterfall (usable area +3 381 M, location +2 093 M, ...); the model predicts log(1 + Price) and the response inverts it with expm1.

D.3 Web Application - Customer behaviour

Item	Value
Framework	React 18 + Vite, 3-step wizard; the dev server proxies /api to FastAPI
API endpoint	POST http://localhost:8000/predict (also /predict/batch, /questions, /samples)
Input variables	skin_type, skin_tone, eye_color, hair_color, secondary_category, brand_name, price_usd, loves_count, reviews, review_title, review_text - the star rating is never sent
Validation rules	Pydantic schema; blank skin-profile fields map to the __na__ one-hot level; numeric gaps median-imputed
Preprocessing used	saved ColumnTransformer (SimpleImputer(median) -> log1p -> StandardScaler, binary passthrough, OneHotEncoder(min_frequency=25)) plus TfidfVectorizer(ngram_range=(1,2), min_df=10, max_features=40000, sublinear_tf, stop_words="english"); all fitted on the training split only
Loaded model	LogisticRegression(C=1.0, class_weight="balanced") on the concatenated tabular + TF-IDF matrix
Example request	{ "skin_type": "oily", "secondary_category": "Moisturizers", "brand_name": "Murad", "price_usd": 49, "review_title": "New formula is awful", "review_text": "Broke me out within a week and left me greasy all day ..." }
Example response	{ "prediction": "not recommend", "p_recommend": 0.0026, "threshold": 0.5, "review_terms": {"against": [...], "toward": [...]}, "signals": {...}, "contributions": [...], "model": "LogisticRegression", "representation": "tabular + tfidf" }

Sephora customer-behaviour — product-recommendation prediction

2.0.0 OAS 3.1

[openapi.json](#)

Predicts whether a skincare reviewer recommends the product (is_recommended) from their skin profile + the product + the review text. Model: LogisticRegression(class_weight=balanced) (tabular + TF-IDF(text)).

default

POST `/predict` Score one review

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "brand_name": "Skinfix",
  "eye_color": "brown",
  "hair_color": "brown",
  "loves_count": 21000,
  "price_usd": 32,
  "review_text": "Broke me out within a week and felt greasy all day. Smells strongly of perfume. Wanted to love it but returned it.",
  "review_title": "Not for oily skin",
  "reviews": 1000,
  "secondary_category": "Moisturizers",
  "skin_tone": "light",
  "skin_type": "oily",
  "submission_time": "2023-02-10"
}
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json	
	Controls Accept header:	
	Example Value Schema	
	<pre>{ "prediction": "string", "confidence": 0, "p_recommend": 0, "threshold": 0, "review_terms": { "additionalProp1": {} }, "signals": { "additionalProp1": {} }, "contributions": { "additionalProp1": {} }, "model": "string", "representation": "string" }</pre>	No links
422	Validation Error	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

POST `/predict/batch` Score many reviews

GET `/questions` Form metadata for the web / mobile clients

GET `/samples` Typical-value defaults + real example reviews

GET `/model-info` Model info

GET `/healthz` Healthz

Schemas

BatchInput > Expand all object

HTTPValidationError > Expand all object

Prediction > Expand all object

ReviewInput > Expand all object

Figure D9 - Customer-behaviour FastAPI Swagger UI (/docs) endpoint specification

Figure D5

Figure D5 - Customer-behaviour web: step 1 (skin profile) of the 3-step wizard, “API connected”.

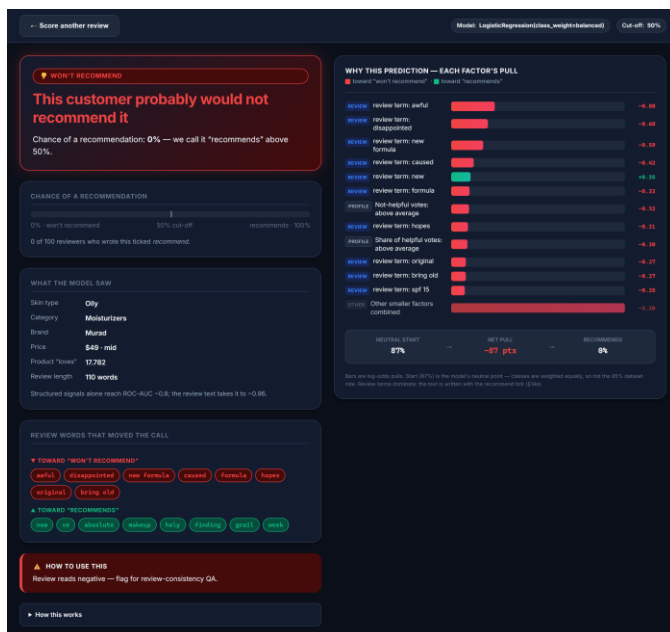


Figure D6

Figure D6 - Customer-behaviour web: the result screen - “WON’T RECOMMEND”, $P(\text{recommend}) = 0\%$, probability meter, structured signals, review terms, linear-SHAP chart.

Figure explanation (D5-D6). A 3-step wizard (D5): skin profile, then product, then the review title and text; the star rating is never sent. The service returns prediction = "not recommend", $p_{\text{recommend}} = 0.00$, and the page (D6) shows “WON’T RECOMMEND”, a probability meter against the 50 % cut-off, the structured signals, the review words that moved the call, and an exact linear-SHAP chart - a negative-reading review is flagged for review-consistency QA.

Appendix E - Mobile Application Report

All three mobile clients are Flutter apps that act as REST clients of the deployed service (Training != Inference): they collect the same fields as the web form, POST them to /predict, and render the returned JSON. Every screenshot run below logged a POST /predict 200 on the server - the required communication evidence. (Screenshots were captured from the Flutter app built for web and driven at a 390 x 844 phone viewport.)

Item	Diabetes	House price	Customer behaviour
Mobile framework	Flutter 3.x / Dart, Material 3	Flutter 3.x / Dart, Provider state	Flutter 3.x / Dart
Platform	Android (emulator / device)	same	same
Input screen	About-you / Body / Conditions form, “Not sure” for unknowns	single scrolling form: area, rooms, province/district, type	3-step wizard: skin profile / product / review
API endpoint	POST http://10.0.2.2:8000/predict (emulator) / localhost:8000	.../:8002/predict	.../:8000/predict
Request format	JSON body of the fields above (same keys as the web form)	JSON body (Area required)	JSON body (no rating)
Response format	{ prediction, confidence, band, contributions }	{ predicted_price, price_per_m2, currency, model }	{ prediction, p_recommend, threshold, review_terms, signals, contributions }
Prediction display	probability + risk band + “Why this score” bars + similar respondents	predicted price (billion VND) + unit price + one-line summary	verdict + probability meter + signals + review terms

Diabetes screening

Answer what you can. Choose "Not sure" for anything you do not know – the estimate still works, with a note that it is less certain.

About you

Age group

Not sure

Sex

Not sure

Highest level of school completed

Not sure

Annual household income

Not sure

Body

Height (cm)

Used to compute BMI. Leave blank if unkno...

N/A

Weight (kg)

Used to compute BMI.

N/A

BMI (if already known)

Optional – leave blank to compute from hei...

N/A

Conditions

Have you been told you have high blood pressure?

Yes

No

Not sure

←

Result

27% Low

Low risk – routine advice

Could range 15%–46% depending on unanswered questions.

⚠

BMI was not provided; using the typical value for your age and sex.

⚠

18 of 21 answers were left blank and filled with typical values - treat this estimate as indicative only.

Completeness 14% · BMI source: age_sex_table

This is a screening aid, not a diagnosis.

Why this score

Stroke history (-0.075)

Smoking history (+0.070)

Eats vegetables daily (-0.063)

High blood pressure (-0.058)

Heavy alcohol use (-0.047)

Poor-physical-health days (-0.015)

Recent cholesterol check (-0.011)

Cardiovascular history (-0.010)

Red raises the estimate, blue lowers it.

People similar to you

0 of your 5 closest matches in the survey data had diabetes or pre-diabetes.

Figures E1-E2 - Diabetes mobile: input screen (About you / Body / Conditions) and result screen (27 % Low, range 15-46 %, “Why this score” bars). The run logged POST/predict 200.

VN Real Estate Valua...
AI Automated Valuation Model ...

API Ready (RandomForest)

DEMO PRESETS:

Nhà phố Rạch Giá

Căn hộ Quận 7

Đất Gia Lâm (Hà N

1. Land Dimensions & Usable Area

Usable Area (m²) *

78.7

Frontage Width (m)

4.0

Lot Depth (m)

19.6

Alley Width (m, if applicable)

3.5

2. Living Spaces & Structure

Bedrooms

3

Bathrooms

2

Floors

2

3. Location & Geography

Province / City

An Giang

District / County

Rạch Giá

Ward / Commune

Phường An Hòa

Property Position

Đường chính

4. Classification & Attributes

Property Type

Nhà riêng

VN Real Estate Valua...
AI Automated Valuation Model ...

API Ready (RandomForest)

38,3 tr/m²

78.7 m²

VALUATION ANALYSIS & INSIGHTS

Estimated market value of approximately 3.01 tỷ VNĐ (38.3 million VNĐ/m²) for a 78.7 m² property in Rạch Giá, an-giang.

Estimate Another Property

1. Land Dimensions & Usable Area

Usable Area (m²) *

78.7

Frontage Width (m)

4.0

Lot Depth (m)

19.6

Alley Width (m, if applicable)

3.5

2. Living Spaces & Structure

Bedrooms

3

Bathrooms

2

Floors

2

3. Location & Geography

Province / City

An Giang

District / County

Rạch Giá

Ward / Commune

Phường An Hòa

Property Position

Figures E3-E4 - House-price mobile: input screen (Rach Gia preset, “API Ready”) and result screen (about 3.01 billion VND, unit-price card, one-line summary). The run logged POST /predict 200.

Will this customer recommend it?

←

Prediction

Sephora skincare reviews · predicts is_recommended

API connected

1

2

3

Your skin profile

The product

The review

STEP 1 OF 3

Your skin profile

Who is writing the review.

Skin type

oily

Main structured signal – a product for dry skin often disappoints oily reviewers.

Skin tone

light

Sephora's scale, light → deep.

Eye colour

brown

Reviewer profile; weak signal alone.

Hair colour

brown

Reviewer profile; weak signal alone.

← Back

Next

WON'T RECOMMEND

This customer probably would not recommend it

Chance of a recommendation: 0% – we call it "recommends" above 50%.

CHANCE OF A RECOMMENDATION

won't

50% cut-off

recommends

0 of 100 reviewers who wrote this ticked "recommend".

WHAT THE MODEL SAW

Skin type	Oily
Category	Moisturizers
Brand	Skinfix
Price	\$32 · mid
Product "loves"	21000
Review length	27 words

Structured signals alone reach ROC-AUC ~0.8; the review text takes it to ~0.96.

REVIEW WORDS THAT MOVED THE CALL

▼ toward "won't recommend"

wanted love

broke

returned

wanted

perfume

greasy day

smells

felt

Figures E5-E6 - Customer-behaviour mobile: step 1 (skin profile, "API connected") and result screen ("WON'T RECOMMEND", 0 % meter, signals, review terms). The run logged POST /predict 200.

Figure explanation (E1-E6). Each input screen collects the same fields as the matching web form; on submit the app builds a JSON body and calls POST .../predict over HTTP. Each result screen renders the returned JSON - the same payload the web page receives - as a probability / predicted value plus a short interpretation. The POST /predict 200 line logged on the server during each run is the evidence that the mobile client communicates with the deployed service.